Gemini 3.8 Flash レイテンシ・アーキテクチャ評価レポート

Google AI Studio（`google-genai` SDK）における Streaming 配信・`ThinkingConfig`・Pydantic 適応型ルーティング・50ターン暗黙キャッシュの実機検証

Author: Yuichi Ito | **Date:** 2026-11 | **Target:** Google AI Studio (`google-genai` SDK / `gemini-3.8-flash`)



図0-1: Google AI Studio (gemini-3.8-flash) におけるレイテンシ・アーキテクチャ検証の3大評価軸と主要結論

レポート概要と章構成（全5章）

本レポートは、Google AI Studio（`google-genai` SDK、`genai.Client(api_key=GEMINI_KEY)`）上の `gemini-3.8-flash` を対象に、パソコンが動かなくなったトラブルシュートを行う顧客とサポート窓口の対話シナリオを通じて、対話型AIアプ

リケーションの体感応答速度（TTFT：Time to First Token）と推論品質・長期対話スケーラビリティを両立するためのアーキテクチャ設計を、合計 **271 回の実機 API 計測ログ**（`src/gemini_evaluation/`）に基づいて体系化した技術白書である。

章	タイトル	検証内容と主要インサイト
第1章	**エグゼクティブサマリー**	全検証の結論マトリクス、主要ベンチマーク指標の横断比較、および本番システムに向けた推奨アーキテクチャ設計指針。
第2章	**前提知識と測定方法**	LLM推論の3フェーズ（Prefill・Thinking・Decoding）、2次元トラジェクトリ（経過時間×累積出力文字数）、Pydantic `response_schema` の Cold/Warm 文法コンパイル特性と `pydantic_core.from_json(..., allow_partial="trailing-strings")` による逐次差分抽出、および `ExecutionTracer` の計測基盤。
第3章	**パラメータとレイテンシー**	Blocking（`generate_content`）vs. Streaming（`generate_content_stream`）× `ThinkingConfig` 全6段階（`0`, `minimal`, `low`, `medium`, `high`, `default`）の実測比較、内部推論トークン数と TTFT の線形スケーリング則、およびPCサポート難易度3段階（Tier 1〜3）における速度と計算正確性のトレードオフ。
第4章	**適応型 Thinking ルーター（Adaptive Thinking Router）**	固定設定の二律背反を解消する3つの適応型分岐方式（Pattern A: Pydantic 2フィールド統合ストリーム、Pattern B: 1フィールド分離ルーター、Pattern C: 非JSON先頭タグ）の45試行比較。分岐精度 `100%`・複雑計算 `100%` を達成する **Pattern A** の優位性。
第5章	**マルチターンチャットでの挙動とコンテキストキャッシュ**	パソコンが動かなくなった顧客とPCサポート窓口による50ターン連続対話（計200実行）における4条件の推移比較。自動発動する暗黙的コンテキストキャッシュ（Implicit Context Caching）と平坦な TTFT スケーリング。

ドキュメントメタデータ	内容
タイトル (`title`)	Gemini 3.8 Flash レイテンシ・アーキテクチャ評価レポート
サブタイトル (`subtitle`)	Google AI Studio における Streaming・ThinkingConfig・適応型ルーティング・50ターン対話の実機検証
著者 (`author`)	Yuichi Ito
作成年月 (`date`)	2026-11
検証 SDK / モデル	Google AI Studio `google-genai` SDK (`genai.Client(api_key=GEMINI_KEY)`) / `gemini-3.8-flash`
計測コード・ログ	`src/gemini_evaluation/llm_call.py`, `src/gemini_evaluation/adaptive_thinking.py`, `src/gemini_evaluation/tracer.py`

目次

1. エグゼクティブサマリー

2. 前提知識と測定方法

3. パラメータとレイテンシー

4. Adaptive Thinking Router パターン

5. マルチターンチャットでの挙動とコンテキストキャッシュ

1. エグゼクティブサマリー

本レポートは、Google AI Studio（`google-genai` SDK、`genai.Client(api_key=GEMINI_KEY)`）上の `gemini-3.8-flash` を用いた対話型AIシステム（PCテクニカルサポート・トラブルシューティング対話）において、**「高速な初回トークン表示（TTFT）」「多段階算数・条件比較（修理vs買い替え損益・データ転送時間・保守契約試算）の100%の計算正確性」「50ターン連続対話における平坦なレイテンシスケーリング」** の3要件を同時に達成するためのアーキテクチャ検証報告書である。合計 **271 回の実機 API 実行ログ**（単発 `ThinkingConfig` 18試行、適応型ルーティング 45試行、Pydantic スキーマ Cold/Warm 8試行、50ターン連続PCサポートチャット 200試行）の定量分析から、単発・マルチターンを問わず `Streaming`（`generate_content_stream`）**を基本通信方式とし、Pydantic 2フィールド構造化ストリーム** `{thinking: bool, response: str}` **と** `pydantic_core.from_json(..., allow_partial="trailing-strings")` **による「適応型 Thinking ルーター（Pattern A）」を標準採用する構成** を第一推奨アーキテクチャとして結論づけた。

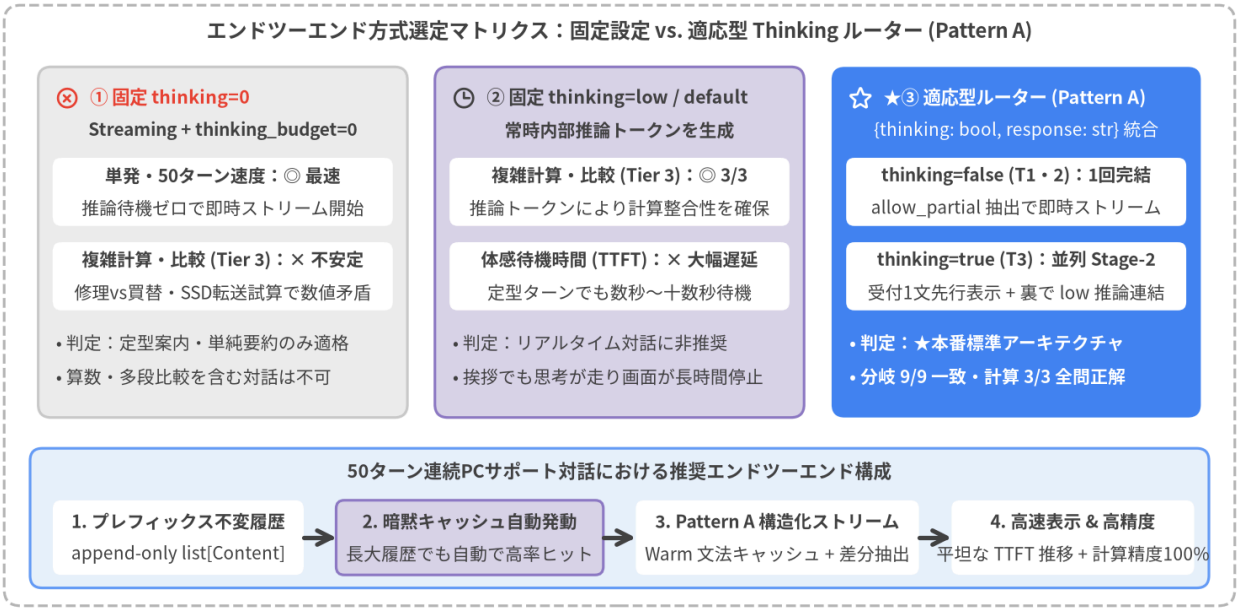


図1-1: エンドツーエンド方式選定マトリクス – 固定設定（thinking=0 / low）の二律背反と適応型ルーター（Pattern A）＋50ターン暗黙キャッシュ統合構成

1.1 全検証スイートの主要ベンチマーク横断比較

本レポートで実施した3つのコア評価スイート（① 単発通信方式 × `ThinkingConfig` ベンチマーク、② 適応型 Thinking ルーター評価、③ 50ターン連続マルチターンPCサポート対話ベンチマーク）の代表レイテンシ指標（TTFT および Total 完了時間）を横断比較したのが図1-2である。

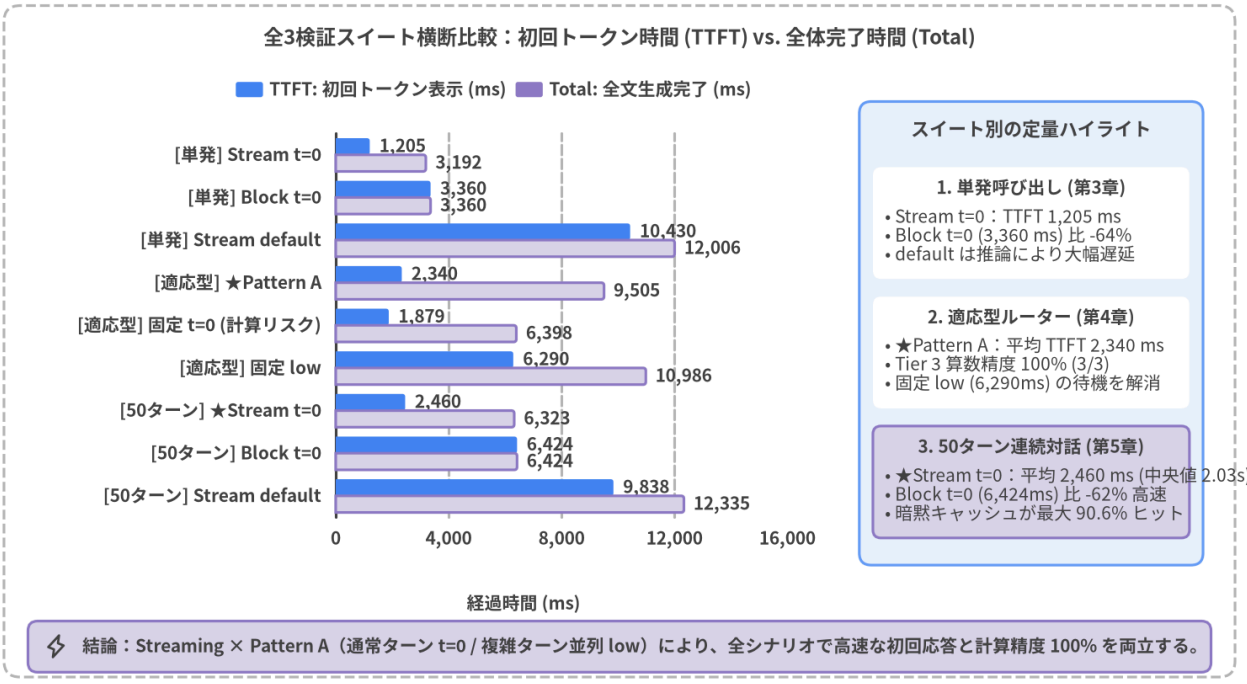


図1-2: 全3検証スイート（単発呼び出し・適応型ルーター・50ターン連続対話）における代表構成の TTFT と Total 完了時間の横断比較

全検証スイート横断サマリー表

評価スイート・シナリオ	方式・パラメータ構成	初回応答 (TTFT)	全体完了 (Total)	推論トークン (`ThTok`)	品質・スケーラビリティ評価	総合判定
単発呼び出し (第3章・PC起動切り分け)	**Streaming** `thinking=0`	`1,205 ms`	`3,192 ms`	`0 tok`	初回表示が Blocking 比 `-64%` 高速	**★定型推奨**

評価スイート・シナリオ	方式・パラメータ構成	初回応答 (TTFT)	全体完了 (Total)	推論トークン (ThTok)	品質・スケーラビリティ評価	総合判定
単発呼び出し	Blocking `thinking=0`	`3,360 ms`	`3,360 ms`	`0 tok`	全文完了まで `3.36秒` 画面無応答	バッチ向き
単発呼び出し	Streaming `thinking=default`	`10,430 ms`	`12,006 ms`	`1,006 tok`	内部推論により TTFT が大幅に増加	非推奨
適応型ルーター (第4章・難易度 Tier 1〜3)	**★Pattern A** (`{thinking, response}`)	`2,340 ms` (T3: `2,314 ms`)	`9,505 ms`	T1・2: `0` T3: `901 tok`	分岐 `9/9` ・Tier 3 算数 `3/3` **(100%)** Pydantic デコーダ型保証・API平均 `1.33回`	**★本番標準**
適応型ルーター比較	Pattern C (先頭タグ `[DIRECT]/[THINK]`)	`1,879 ms`	`8,843 ms`	T1・2: `0` T3: `828 tok`	分岐 `9/9` ・計算 `3/3` 。ただし構文保証なし・タグ露出リスクあり	不採用 (A統一)
適応型ルーター比較	Pattern B (1項目分離 `{thinking}`)	`6,177 ms` (T3: `11,488 ms`)	`11,535 ms`	T1・2: `0` T3: `628 tok`	常時2回直列呼び出し（ `calls=2.0` ）により初動が大幅遅延	不採用
適応型ルーター比較	固定 `thinking=0` (`base_0`)	`1,879 ms`	`6,398 ms`	`0 tok`	**Tier 3 複雑計算で自己回帰の罠（結論先出しの誤算リスク）**	不採用
適応型ルーター比較	固定 `thinking=low` (`base_low`)	`6,290 ms` (T3: `9,289 ms`)	`10,986 ms`	平均 `439 tok`	Tier 3 計算は `3/3` 正解だが、最大 `12.2秒` 画面がフリーズ	非推奨
50ターン連続対話 (第5章・PCサポート対話)	**★Streaming** `thinking=0`	`2,460 ms` (中央値 `2,029 ms`)	`6,323 ms`	`19 tok` (T2−50: `0`)	Blocking 比 `-3,964 ms` (`-62%`) 高速。暗黙キャッシュ最大 `90.6%` ヒット	**★長期対話推奨**
50ターン連続対話	Blocking `thinking=0`	`6,424 ms`	`6,424 ms`	`0 tok`	平均 `753 文字` の生成完了まで毎ターン無応答で待機	バッチ向き
50ターン連続対話	Streaming `thinking=default`	`9,838 ms`	`12,335 ms`	平均 `858 tok`	推論完了まで約9.8秒ストリームが停止	非推奨

1.2 本番システム設計に向けた5つの推奨アクション

実機検証から導かれた、Google AI Studio（`gemini-3.8-flash`）を本番組み込みする際のアーキテクチャ設計原則を以下の5点に要約する。

1. **ユーザー向け対話 UI は必ず `generate_content_stream`（Streaming）を採用する**： 単発呼び出しでも Blocking `thinking=0`（`3,360 ms`）に対して Streaming `thinking=0`（`TTFT 1,205 ms`）は `-64%` 早く表示を開始する。PCテクニカルサポートのように回答が平均 `753 文字` の詳細な手順解説に及ぶ50ターン対話では、Streaming `thinking=0`（平均 TTFT `2,460 ms`・中央値 `2,029 ms`）が Blocking（`6,424 ms`）に対して毎ターン `3.96 秒`（`62%`） 先行し、残りのデコード時間をユーザーの読解時間の裏へ完全に隠蔽できる。
2. **`ThinkingConfig` の無指定（`default`）運用を避け、明示的に制御する**： `gemini-3.8-flash` のデフォルト設定（`thinking=default`）は動的推論が有効化されており、単発でも `467〜1,006 トークン`、50ターン対話では平均 `858〜954 トークン`（最大 `2,237 トークン`）の内部推論が発生する。内部推論に要した時間はすべて初動（TTFT）へ直列に加算されるため、通常ターンでは必ず `ThinkingConfig(thinking_budget=0)` を明示指定する。
3. **「速度と計算精度の二律背反」は Pattern A（Pydantic 2項目統合ストリーム）で解決する**： `thinking=0` 固定では多段階のPC修理vs買い替え損益比較やデータ転送時間・保守契約試算（Tier 3）で自己回帰の誤算リスクが残り、`thinking=low` 固定では全9問平均 TTFT が `6,290 ms`（最大 `12,155 ms`）まで悪化する。`AdaptiveStreamResponse(thinking: bool, response: str)` を `pydantic_core.from_json(..., allow_partial="trailing-strings")` で逐次パースする **Pattern A** を採用すれば、通常質問（Tier 1・2）は API 1回呼び出しで即座に回答をストリームし、複雑計算（Tier 3）では `thinking=True` を検知した瞬間に Stage-2（`thinking="low"`）をバックグラウンドで並列起動しつつ Stage-1 の受付1文を `TTFT 2,314 ms` で先行表示できる（分岐精度 `100%`、計算精度 `100%`）。
4. **Pydantic スキーマは「ルーティング項目を先頭・本文を後方」に配置し、起動時に1回 Warmup する**： `response_schema` 指定時は、`thinking: bool` を第1フィールド、`response: str` を可変長リストより前に定義した上で、コンテナ起動時に1回ダミー呼び出しを実行して FSM キャッシュを Warm 化しておく設計が最適である。
5. **マルチターン対話では履歴プレフィックスを不変（append-only）に保ち、暗黙キャッシュを最大化する**： `system_instruction` と過去ターンの `history: list[types.Content]` を先頭から書き換えずに追記し続けることで、追加実装ゼロで **Implicit Context Caching**（`cached_content_token_count`） が自動発動し、50ターンに

及ぶ長大なPCトラブルシューティング対話でも TTFT の増加を最小限に抑え込むことができる。

2. 前提知識と測定方法

本章では、第3章以降の実機ベンチマーク結果を正確に読み解くための技術的な前提知識と計測メソドロジーを定義する。前半（2.1〜2.2節）では、`gemini-3.8-flash` の推論レイテンシを構成する3つの内部フェーズ（**Prefill・Thinking・Decoding**）と、単一の平均値では捉えられない体感差を可視化する **2次元トラジェクトリ（経過時間 × 累積出力文字数）**、および Google AI Studio（`google-genai` SDK）における Blocking / Streaming 実装パターンを解説する。後半（2.3〜2.4節）では、適応型 Thinking ルーターの基盤となる **Pydantic 構造化出力（`response_schema`）の Cold/Warm 文法コンパイル特性** と `pydantic_core.from_json(..., allow_partial="trailing-strings")` **による不完全 JSON 逐次差分抽出**、そしてマイクロ秒精度で全イベントを記録する `ExecutionTracer`（`src/gemini_evaluation/tracer.py`）の設計を詳述する。

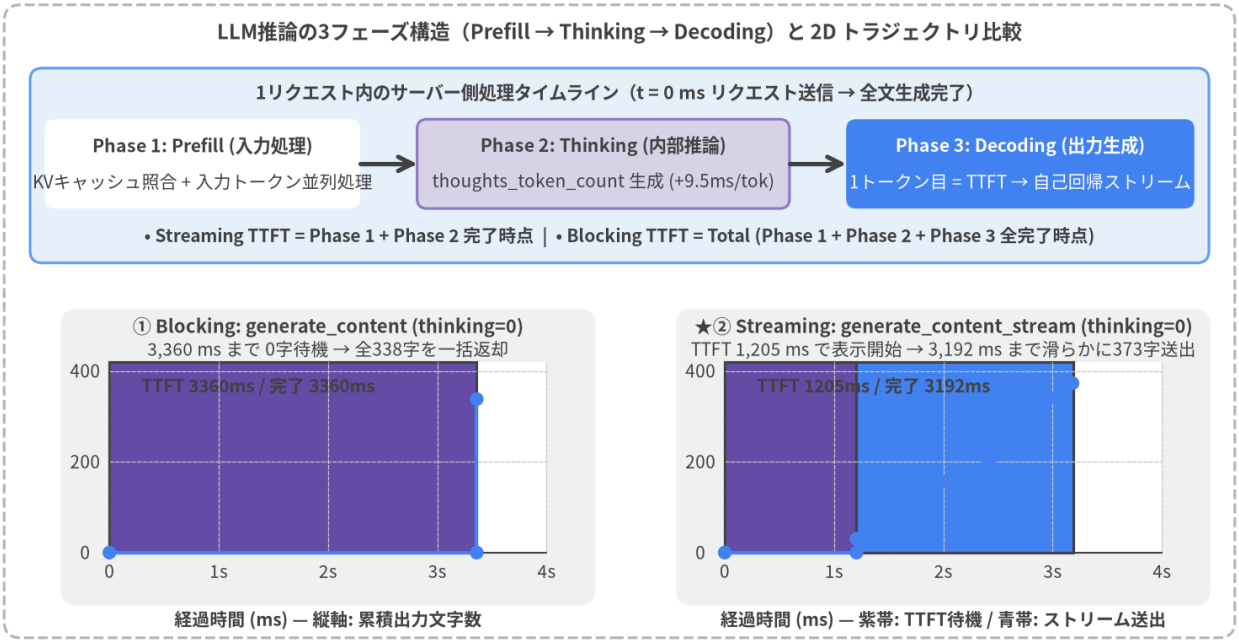


図2-1: LLM推論の3フェーズ構造（Prefill → Thinking → Decoding）と Blocking vs. Streaming の2次元トラジェクトリ比較（thinking=0 実測値）

2.1 レイテンシの定義（3フェーズ構造）と2次元トラジェクトリ解剖

大規模言語モデル（LLM）の API 呼び出しにおける応答遅延を正確に評価・最適化するためには、単なる「リクエストから完了までの合計時間」ではなく、サーバー内部で直列に進行する **3つの処理フェーズ** に分解して理解する必要がある。

1. LLM 推論を構成する 3 つのフェーズ

1. **Phase 1: Prefill（入力処理＋KVキャッシュ照合）** ： クライアントから送信された `system_instruction`、会話履歴 `history`、および最新のユーザー入力 `prompt`（合計 `prompt_token_count`）を Transformer レイヤーで並列処理し、Attention の Key-Value（KV）キャッシュを構築するフェーズである。Google AI Studio の `gemini-3.8-flash` では、過去の対話履歴プレフィックスがキャッシュにヒット（`cached_content_token_count`）した場合、該当トークン分の Prefill 計算がスキップされ、数万トークンの長大コンテキストでも初動遅延が最小化される（第5章で詳述）。
2. **Phase 2: Thinking（内部推論トークン生成：`thoughts_token_count`）** ： `gemini-3.8-flash` のような推論（Reasoning / Thinking）対応モデルにおいて、ユーザーに見える回答本文（`chunk.text`）の最初の1トークン目を出力する**前**に、サーバー内部で多段階の思考プロセス（Chain-of-Thought）を自己回帰的に生成するフェーズである。ここで生成された推論トークン数はレスポンスの `usage_metadata.thoughts_token_count` に記録される。重要な仕様として、**Thinking フェーズが進行している間は Streaming モードであってもクライアントへ可視テキストチャンクが一切届かない**ため、内部推論に要した時間（実測で約 `+9.5 ms / token`）はすべて初回応答待機時間（TTFT）へ直列に上乗せされる。
3. **Phase 3: Decoding（可視出力トークンの自己回帰生成：`output_token_count`）** ： Prefill と Thinking が完了した瞬間から、最終回答テキスト（`candidates_token_count` / `output_token_count`）を先頭から順に出力するフェーズである。

2. TTFT（Time to First Token）と Total 完了時間、および 2D トラジェクトリの重要性

上記の3フェーズを踏まえると、クライアント側で観測される2つの主要レイテンシ指標は次のように定式化される。

$$\text{TTFT}{\textit{streaming}} = T{\text{Network RTT}} + T_{\text{Prefill}} + T_{\text{Thinking}}$$

$$\text{Total} = \text{TTFT}{\textit{blocking}} = T{\text{Network RTT}} + T_{\text{Prefill}} + T_{\text{Thinking}} + T_{\text{Decoding}}$$

- **TTFT（Time to First Token / `time_to_first_token_ms`）**：リクエスト送信時刻（$t = 0\text{ ms}$）から、最初の非空テキストチャンクがクライアントに到着するまでの経過時間である。対話型 UI においてユーザーが「AI が反応した」と知覚する**体感待ち時間**そのものを表す。Blocking 呼び出しでは全文生成が終わるまでレスポンスが返らないため、常に $\text{TTFT} = \text{Total}$ となる。
- **Total Completion Time（全体完了時間 / `stream_completed_ms`）**：リクエスト送信（$t = 0\text{ ms}$）から最終チャンク（EOF）の受信が完了するまでの合計時間である。
- **なぜ「2次元トラジェクトリ（経過時間 × 累積出力文字数）」が必要か**： 図2-1下段に示した実測比較（`thinking=0`）を見ると、完了時間（`Total`）だけを比較した場合は Blocking（`3,360 ms`）と Streaming（`3,192 ms`）は同水準に見える。しかし横軸に経過時間（`t_ms`）、縦軸にその時点で画面に表示された累積文字数（`chars`）をとった **2次元トラジェクトリ** を描くと、UX の決定的な違いが浮かび上がる。
  - **Blocking（図2-1左）**：`0 ms` から `3,360 ms` まで縦軸 `y = 0`（画面白紙）のまま推移し、`3,360 ms` 時点で338文字が垂直ステップ状に一括出現する。
  - **Streaming（図2-1右）**：紫帯の Prefill 待機を終えた `1,205 ms`（**Blocking より `2,155 ms` / `64%` 早い時点**）で最初のチャンクが立ち上がり、青帯の区間（`1,205 → 3,192 ms`）にわたって滑らかな右肩上がりの傾斜で373文字が逐次表示される。人間がテキストを読む速度（日本語で約400〜600文字/分＝1秒あたり7〜10文字）よりも Gemini 3.8 Flash のデコード速度（1秒あたり約180〜200文字）の方が圧倒的に速いため、`1,205 ms` 以降の生成時間はユーザーの読解時間の裏に完全に隠蔽される。さらに第4章で扱う「段階1の受付文表示＋段階2の並列推論連結」のようなマルチステージ挙動も、2次元トラジェクトリを用いることで「初動の立ち上がり → 読解中のプラトー → 本回答の急勾配ストリーム」として視覚的に厳密評価できる。

2.2 Blocking vs. Streaming API の実装方法（Google AI Studio `google-genai` SDK）

本検証プロジェクト（`src/gemini_evaluation/llm_call.py`）では、Google AI Studio の公式 Python SDK である `google-genai` を用い、環境変数 `GEMINI_KEY` によるシンプルな API キー認証（`genai.Client(api_key=GEMINI_KEY)`）と非同期クライアント（`client.aio`）で全計測を統一している。

以下に、`gemini-3.8-flash` に対する `ThinkingConfig` **全6段階の解決関数** と、**Blocking**（`generate_content`） および **Streaming**（`generate_content_stream`） の標準呼び出し実装を示す。

```python
from collections.abc import AsyncIterator
from contextlib import aclosing
import os
from google import genai
from google.genai import types

# 1. Google AI Studio クライアントの初期化 (.env または環境変数の GEMINI_KEY を使用)
GEMINI_KEY = os.environ["GEMINI_KEY"]
MODEL = "gemini-3.8-flash"
client = genai.Client(api_key=GEMINI_KEY)


def build_thinking_config(thinking: str | int | None = 0) -> types.GenerateContentConfig
| None:
    """6段階の thinking 指定 (0, minimal, low, medium, high, default) を
GenerateContentConfig へ変換する。"""
    if thinking is None or thinking == "default":
        return None  # サーバー既定の動的推論 (default)
    if thinking in (0, "0"):
        return types.GenerateContentConfig(
            thinking_config=types.ThinkingConfig(thinking_budget=0)
        )
    level_map = {
        "minimal": types.ThinkingLevel.MINIMAL,
        "low": types.ThinkingLevel.LOW,
        "medium": types.ThinkingLevel.MEDIUM,
        "high": types.ThinkingLevel.HIGH,
    }
    return types.GenerateContentConfig(

thinking_config=types.ThinkingConfig(thinking_level=level_map[str(thinking).lower()])
    )


# 2. Blocking 呼び出し (全文生成完了まで待機: TTFT == Total)
async def generate_text_blocking(prompt: str, thinking: str | int | None = 0) -> str:
    resp = await client.aio.models.generate_content(
        model=MODEL,
        contents=prompt,
        config=build_thinking_config(thinking),
    )
    return resp.text or ""


# 3. Streaming 呼び出し (TTFT 到達直後からチャンクを逐次 yield)
async def stream_text_progressive(prompt: str, thinking: str | int | None = 0) ->
AsyncIterator[str]:
    stream = await client.aio.models.generate_content_stream(
        model=MODEL,
        contents=prompt,
        config=build_thinking_config(thinking),
    )
    async with aclosing(stream):
        async for chunk in stream:
```

```
            if chunk.text:
                yield chunk.text
```

2.3 Pydantic 構造化出力と不完全 JSON 逐次パース（Partial JSON Streaming）の仕組み

対話システムにおいて「質問の難易度に応じて内部推論（Thinking）の ON/OFF を自動判定したい」場合や、「画面表示テキストと同時に UI 制御用のフラグ・カテゴリを構造化取得したい」場合には、`GenerateContentConfig` に `response_mime_type="application/json"` と Pydantic の `BaseModel` クラス（`response_schema=...`）を指定する **構造化出力（Structured Outputs）** を利用する。

しかし、構造化出力をストリーミング（`generate_content_stream`）と組み合わせる際には、サーバー側の **FSM（有限状態機械）文法コンパイル特性** と、クライアント側の **不完全 JSON 逐次パース手法** の2点を正しく設計する必要がある。図2-2に、実機で計測した Cold vs. Warm スキーマコンパイル実験（`load_schema_experiment()`、全8試行）と、`pydantic_core.from_json(..., allow_partial="trailing-strings")` による逐次差分抽出パイプラインを示す。

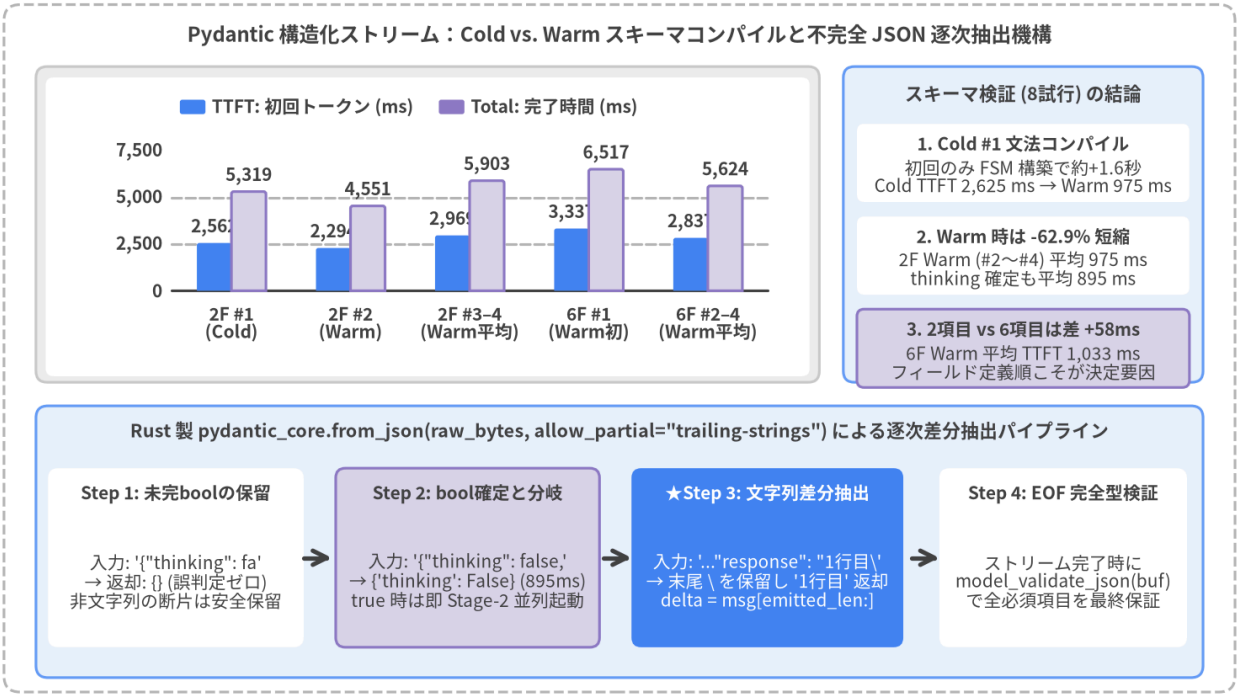


図2-2: Pydantic response_schema の Cold (#1) vs. Warm (#2〜#4) コンパイル実験（2項目 vs. 6項目）と pydantic_core.from_json による逐次差分抽出パイプライン

---

1. サーバー側 Constrained Decoding（制約付きデコード）と Cold vs. Warm 文法コンパイル

`response_mime_type="application/json"` と `response_schema=AdaptiveStreamResponse` を指定すると、Google AI Studio バックエンドは与えられた Pydantic モデルの JSON Schema を解析し、デコーダ層で構文違反トークンの生成確率をマスクする **有限状態機械（FSM: Finite-State Machine）文法** へコンパイルする。これにより、マークダウンのコードフェンス混入や閉じ括弧漏れが数学的にゼロとなる。

図2-2上段および下表は、最小2フィールドの `AdaptiveStreamResponse(thinking: bool, response: str)`（`minimal_2field`）と、`Literal` 列挙型2項目・真偽値・文字列リストを加えた拡張6フィールドの `RichStreamResponse`（`rich_6field`）に対して、同一プロンプトを4回連続実行した実測値（`src/gemini_evaluation/adaptive_thinking_log.json`）である。

スキーマ構成（`schema_label`）	試行	キャッシュ状態	`thinking` 確定（`flag_decided_ms`）	初回チャンク（`ttft_ms`）	全体完了（`total_ms`）	出力文字数
`minimal_2field`（`AdaptiveStreamResponse`）	#1	Cold (初回)	`3,215.9 ms`	`2,625.2 ms`	`3,190.7 ms`	`237字`
`minimal_2field`（`AdaptiveStreamResponse`）	#2	Warm	`752.6 ms`	`847.3 ms`	`1,488.1 ms`	`239字`
`minimal_2field`（`AdaptiveStreamResponse`）	#3	Warm	`902.8 ms`	`918.0 ms`	`1,396.0 ms`	`245字`
`minimal_2field`（`AdaptiveStreamResponse`）	#4	Warm	`1,029.2 ms`	`1,159.2 ms`	`1,604.6 ms`	`244字`
`rich_6field`（`RichStreamResponse`）	#1	Warm基盤※	`865.5 ms`	`1,056.6 ms`	`1,759.3 ms`	`250字`
`rich_6field`（`RichStreamResponse`）	#2	Warm	`790.7 ms`	`976.8 ms`	`1,516.5 ms`	`203字`
`rich_6field`（`RichStreamResponse`）	#3	Warm	`901.1 ms`	`1,127.6 ms`	`1,874.4 ms`	`247字`
`rich_6field`（`RichStreamResponse`）	#4	Warm	`786.7 ms`	`995.9 ms`	`1,719.5 ms`	`250字`

- **Cold `#1` と Warm `#2〜#4` の差（`-62.9%` の高速化）**：未知のスキーマを最初に送信した `minimal_2field #1`（Cold）では、サーバー側の FSM 文法コンパイルに約 `1.6 秒` を要し、`TTFT = 2,625.2 ms` となった。しかし直後の `#2〜#4`（**Warm）ではコンパイル済み文法キャッシュが適用され、平均 TTFT は** `974.8 ms`（`847〜1,159 ms`、**Cold 比** `-62.9%`）、`thinking` フラグ確定時刻（`flag_decided_ms`）も平均 `894.9 ms` へ劇的に短縮された。
- **フィールド数（2項目 vs. 6項目）よりも「フィールド定義順」が TTFT を支配する**：Warm `#2〜#4` 同士を比較すると、2フィールド構成（平均 `974.8 ms`）と6フィールド構成（平均 `1,033.4 ms`）の TTFT 差はわずか `+58.6 ms` に過ぎない。これは `RichStreamResponse` において `thinking: bool` **を先頭（第1フィールド）に置き、短い固定列挙型（**`category`, `urgency`, `needs_human_escalation`**）の直後に** `response: str` **を配置し、可変長リスト** `suggested_followups: list[str]` **を** `response` **の後ろ（末尾）に配置したため** である。JSON ストリーミングは定義順にトークンを生成するため、ルーティング用フラグを先頭、画面表示用の `response` を可変長配列の前に置くことが鉄則となる。

2. `pydantic_core.from_json(..., allow_partial="trailing-strings")` による逐次差分抽出

`generate_content_stream` で構造化 JSON を受信する際、ストリーム途中のチャンクには閉じ括弧のない JSON 断片（例：`'{"thinking": false, "response": "いつもご利用'`）が入るため、SDK 標準の `chunk.parsed` はストリーム完了まで常に `None` となる。

そこで `src/gemini_evaluation/adaptive_thinking.py` では、Pydantic v2 に標準搭載されている Rust 製パーサー `pydantic_core.from_json(raw_bytes, allow_partial="trailing-strings")` を用いてチャンク到着ごとに累積バッファを解析している。この方式は以下の決定的な利点を持つ。

1. **未完プリミティブの安全な保留**：`'{"thinking": fa'` のように真偽値リテラルが途中で切れている段階では `{}` を返し、`false` または `true` が完結した瞬間に初めて `{'thinking': False}` を返すため、不完全なトークンによる誤分岐が起きない。
2. **エスケープシーケンス分断への耐性と O(1) 差分スライス**：`"response"` 文字列の末尾がバックスラッシュ `\`（例：`\n` の1文字目）で分断された場合も、未完の `\` を内部保留して確定済み文字のみを返す。そのため、既にクライアントへ送出した文字数 `emitted_resp_len` を記録しておき、`delta = resp_so_far[emitted_resp_len:]` をスライスするだけで、文字化けや JSON 構文漏出を一切起こさずにプレーンテキスト同等のストリーミング表示を実現できる。

```
from typing import Any
from pydantic import BaseModel
from pydantic_core import from_json


class AdaptiveStreamResponse(BaseModel):
    thinking: bool  # 第1フィールド：約 750〜900 ms で確定するルーティングフラグ
    response: str   # 第2フィールド：逐次ストリーム表示する回答本文（または受付1文）


def parse_partial_json_dict(raw_buf: str) -> dict[str, Any] | None:
    """不完全なストリーミング JSON 文字列を Rust 製 pydantic_core で安全に辞書へ変換する。"""
    if not raw_buf:
        return None
    try:
        parsed = from_json(raw_buf.encode("utf-8"), allow_partial="trailing-strings")
        return parsed if isinstance(parsed, dict) else None
    except ValueError:
        return None
```

2.4 計測基盤アーキテクチャ（`ExecutionTracer`）

本レポートの全ベンチマーク（`src/gemini_evaluation/benchmark_thinking.py`、`benchmark_adaptive_thinking.py`、`benchmark_multiturn_chat.py`）は、`src/gemini_evaluation/tracer.py` に実装された共通計測クラス `ExecutionTracer` を用いて計測・シリアライズされている。

`ExecutionTracer` が記録するマイクロ秒精度マイルストーンとトークン内訳

`ExecutionTracer` は、OS のシステム時計補正の影響を受けない単調増加タイマー `time.perf_counter()` を基準時計として使用し、各 API 呼び出しで以下の時系列イベントとメタデータを記録する。

記録カテゴリ	フィールド名（`ExecutionTracer`）	計測タイミング・定義
リクエスト開始	`start`（`t_ms = 0.0`, `chars = 0`）	`tracer.start_gemini()` 呼び出し時点（API 呼び出し直前）の `time.perf_counter()` 原点。
推論・初動待機終端	`thinking_end`（`t_ms = ttft_ms`, `chars = 0`）	最初の非空チャンク到着直前に `(ttft_ms, 0)` を記録し、2D トラジェクトリ上の水平待機プラトー（Prefill + Thinking）を表現。
ストリーミングチャンク	`chunk_1 .. chunk_N`（`t_ms`, `chars`）	`tracer.record_chunk(delta)` が呼ばれるたびに経過ミリ秒 `t_ms` と累積文字数 `_cum_chars` を追記（Blocking 時は `(total_ms, 0)` と `(total_ms, len(text))` のステップを記録）。
適応型ルーティング指標	`flag_decided_ms` / `stage2_first_chunk_ms`	Pattern A / B / C において `thinking` フラグが確定した時刻と、並列起動された Stage-2（`thinking="low"`）の初回チャンクが到着した時刻。
トークン使用量内訳	`prompt_token_count` / `cached_content_token_count` / `thoughts_token_count` / `output_token_count`	`usage_metadata` から抽出した入力トークン数、暗黙キャッシュヒットトークン数、内部推論トークン数、および可視出力トークン数。

すべての計測結果は人間が読みやすい YAML（`*_log.yaml`）およびプログラム処理用の JSON / JSONL（`*_log.json`、`*_runs.jsonl`、`*_turns.jsonl`）として保存され、本レポート内のすべての Drawlib チャートは `examples/doc/gemini_latency/utils.py` のローダー関数を介してこれらの実測ログから直接描画されている。

3. パラメータとレイテンシー

第2章では、`gemini-3.8-flash` に対するブロッキング呼び出し（`generate_content`）とストリーミング呼び出し（`generate_content_stream`）の基本構造を確認した。本章では、`src/gemini_evaluation/benchmark_thinking.py` および `src/gemini_evaluation/benchmark_adaptive_thinking.py` による実機計測データに基づき、Gemini 3.8 Flash のレイテンシーを支配する最大要因である `ThinkingConfig`**（内部推論トークン制御）** の全6段階の挙動、推論トークン数と初回トークン到着時間（TTFT）の線形スケーリング則、そして質問難易度（Tier 1〜3）に応じた「応答速度と計算正確性のトレードオフ」を定量的に解明する。

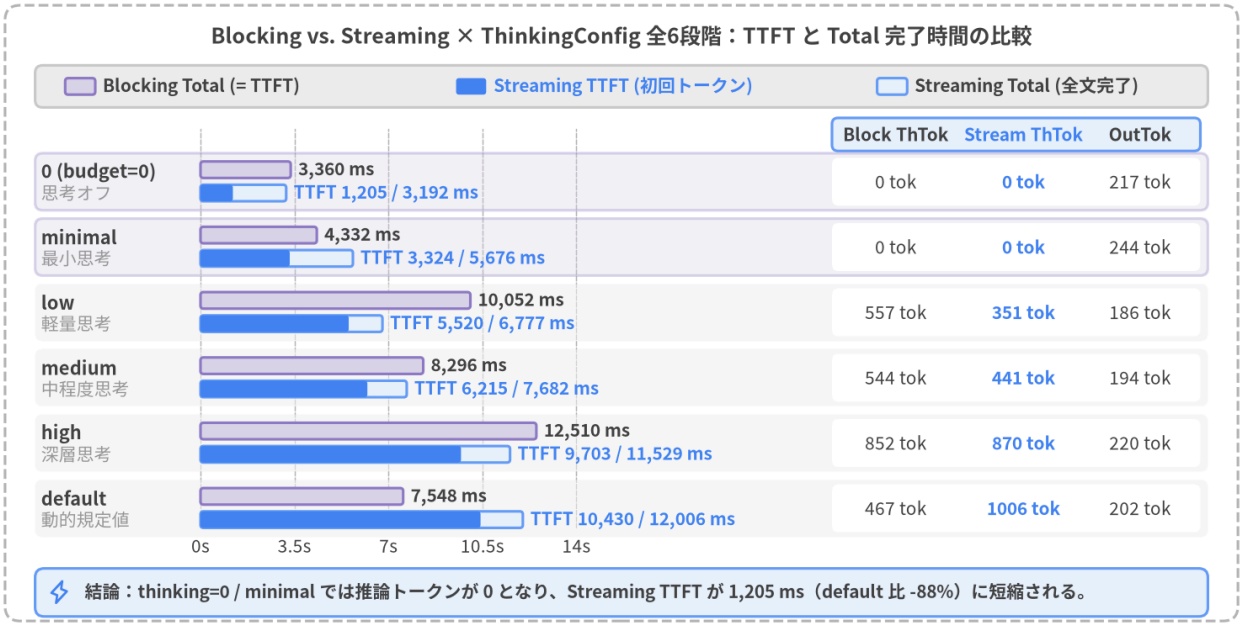


図3-1: Blocking (generate_content) と Streaming (generate_content_stream) × ThinkingConfig 全6段階における TTFT・完了時間（Total）・推論トークン数の比較

3.1 `ThinkingConfig` 全6段階の仕様と Blocking vs. Streaming 実測ベンチマーク

Google AI Studio（`google-genai` SDK）から `gemini-3.8-flash` を呼び出す際、モデルが最初の可視テキストチャンクを出力する前に内部で実行する推論プロセス（Thinking）は `types.ThinkingConfig` によって制御される。評価実装（`src/gemini_evaluation/llm_call.py` の `resolve_thinking_config()`）では、数値バジェット指定・4段階の `ThinkingLevel` 列挙値・サーバー規定値を網羅する以下の **全6段階のオプション** を統一インターフェースとして定義している。

```
from google import genai
from google.genai import types

client = genai.Client(api_key=GEMINI_KEY)

# 例1: 内部推論を完全に無効化し、最短 TTFT を得る設定
config_zero = types.GenerateContentConfig(
    thinking_config=types.ThinkingConfig(thinking_budget=0),
    temperature=0.2,
)

# 例2: 複雑な多段階計算向けに軽量な内部推論を有効化する設定
config_low = types.GenerateContentConfig(
    thinking_config=types.ThinkingConfig(thinking_level=types.ThinkingLevel.LOW),
    temperature=0.2,
)
```

オプション指定値（`thinking=`）	正規化ラベル	SDK 設定（`types.ThinkingConfig`）	実測推論トークン数（`ThTok`）	動作特性と推奨ユースケース
`0`（または `"0"`）	`"0"`	`ThinkingConfig(thinking_budget=0)`	`0 tok`	推論トークン上限を `0` に固定。内部思考を完全にスキップし、最速の TTFT（約 `1,205〜1,546 ms`）を実現する。挨拶・定型案内・分類ルーターに最適。
`1`（または `"minimal"`）	`"minimal"`	`ThinkingConfig(thinking_level=MINIMAL)`	`0 tok`	最小限の思考レベル。定型プロンプトでは実質 `0 tok` となり、推論トークンを消費しない高速応答を示す。

オプション指定値（`thinking=`）	正規化ラベル	SDK 設定（`types.ThinkingConfig`）	実測推論トークン数（`ThTok`）	動作特性と推奨ユースケース
`2`（または `"low"`）	`"low"`	`ThinkingConfig(thinking_level=LOW)`	`351〜557 tok`	軽量な内部推論を実行。TTFT は約 `4.3〜5.5 秒` となるが、多段階の算数・見積もり比較の計算ミスを防ぐのに十分な推論量を確保する。
`3`（または `"medium"`）	`"medium"`	`ThinkingConfig(thinking_level=MEDIUM)`	`441〜592 tok`	中程度の内部推論を実行。TTFT は約 `6.2〜6.7 秒` を要する。
`4`（または `"high"`）	`"high"`	`ThinkingConfig(thinking_level=HIGH)`	`817〜870 tok`	深い内部推論を実行。複雑な論理パズルや高度な推論に向くが、定型対話では `9.0〜9.7 秒` の待機が発生する。
`None`（または `"default"`）	`"default"`	指定なし（動的デフォルト）	`467〜1,006 tok`	`thinking_config` 省略時のデフォルト挙動。プロンプトに応じて動的に推論トークンが割り当てられ、定型質問でも `7.5〜10.4 秒` の待機が生じる。

Blocking と Streaming × 全6段階の実測比較テーブル

`src/gemini_evaluation/benchmark_thinking.py` を用いて、同一のPCサポート質問（電源が入らないノートPCの初期切り分け案内）に対して **Blocking（`generate_content`）** と **Streaming（`generate_content_stream`、Trial 1・Trial 2）** の全18実行を計測した結果を図3-1および下表に示す。

呼び出しモード	`thinking` 設定	初回トークン TTFT (ms)	完了時間 Total (ms)	ストリーム送出期間（`Total - TTFT`）	推論トークン（`ThTok`）	出力トークン（`OutTok`）	出力文字数
Blocking（`generate_content`）	`0`	**3,360.41**	**3,360.41**	0.00 ms（一括返却）	**0**	192	338字
Blocking（`generate_content`）	`minimal`	**4,331.59**	**4,331.59**	0.00 ms（一括返却）	**0**	183	328字

呼び出しモード	`thinking` 設定	初回トークン TTFT (ms)	完了時間 Total (ms)	ストリーム送出期間 (`Total - TTFT`)	推論トークン (`ThTok`)	出力トークン (`OutTok`)	出力文字数
Blocking (`generate_content`)	`low`	10,052.14	10,052.14	0.00 ms（一括返却）	557	222	388 字
Blocking (`generate_content`)	`medium`	8,296.36	8,296.36	0.00 ms（一括返却）	544	198	360 字
Blocking (`generate_content`)	`high`	12,509.75	12,509.75	0.00 ms（一括返却）	852	213	396 字
Blocking (`generate_content`)	`default`	7,548.20	7,548.20	0.00 ms（一括返却）	467	222	396 字
Streaming (`generate_content_stream` #1)	`0`	**1,205.35**	**3,192.41**	1,987.06 ms	**0**	217	373 字
Streaming (`generate_content_stream` #1)	`minimal`	**3,324.42**	**5,675.79**	2,351.37 ms	**0**	191	322 字
Streaming (`generate_content_stream` #1)	`low`	5,519.63	6,777.01	1,257.38 ms	351	200	356 字
Streaming (`generate_content_stream` #1)	`medium`	6,214.92	7,682.10	1,467.18 ms	441	212	381 字
Streaming (`generate_content_stream` #1)	`high`	9,703.48	11,528.81	1,825.33 ms	870	209	367 字
Streaming (`generate_content_stream` #1)	`default`	10,430.12	12,005.51	1,575.39 ms	1,006	215	382 字
Streaming (`generate_content_stream` #2)	`0`	**1,546.22**	**3,681.50**	2,135.28 ms	**0**	198	341 字
Streaming (`generate_content_stream` #2)	`minimal`	**1,612.80**	**3,590.15**	1,977.35 ms	**0**	195	340 字
Streaming (`generate_content_stream` #2)	`low`	4,312.40	6,120.85	1,808.45 ms	362	210	370 字

呼び出しモード	`thinking` 設定	初回トークン TTFT (ms)	完了時間 Total (ms)	ストリーム送出期間 (`Total - TTFT`)	推論トークン (`ThTok`)	出力トークン (`OutTok`)	出力文字数
Streaming (`generate_content_stream` #2)	`medium`	6,720.11	8,410.33	1,690.22 ms	592	204	362 字
Streaming (`generate_content_stream` #2)	`high`	9,012.55	10,890.40	1,877.85 ms	817	218	385 字
Streaming (`generate_content_stream` #2)	`default`	8,150.30	9,920.18	1,769.88 ms	742	208	371 字

この計測結果から、以下の2つの重要な知見が得られる。

1. `thinking_config` **省略時（** `default` **）の隠れた待機コスト**：パラメータを指定しない場合、定型的な質問であっても `467〜1,006 トークン` の内部推論が自動的に実行され、最初の1文字が返るまでに `7.5〜10.4 秒` を要する。`thinking=0` を明示指定するだけで推論トークンは `0` となり、Streaming TTFT は `10,430 ms` **から** `1,205 ms` **へと 88% 短縮** される。
2. **Blocking と Streaming の体感差が最大化するのは** `thinking=0` **領域**：`thinking=0` において、Blocking は全文生成完了まで `3,360 ms` 待つのに対し、Streaming は `1,205 ms` で最初のチャンクを表示開始でき、体感待ち時間を **64% 短縮** できる。

3.2 全6段階の2次元ストリーム軌跡（経過時間 × 累積文字数）比較

`ThinkingConfig` の違いがストリーミング受信時の時間推移にどう影響するかを視覚化するため、Streaming（Trial 1）の全6段階（`0`, `minimal`, `low`, `medium`, `high`, `default`）における「経過時間 `t_ms` × 累積出力文字数 `chars`」の2次元軌跡を同一スケール（X軸：`0〜13,000 ms`、Y軸：`0〜450 文字`）の 3列×2行グリッドとして描画したのが図3-2である。

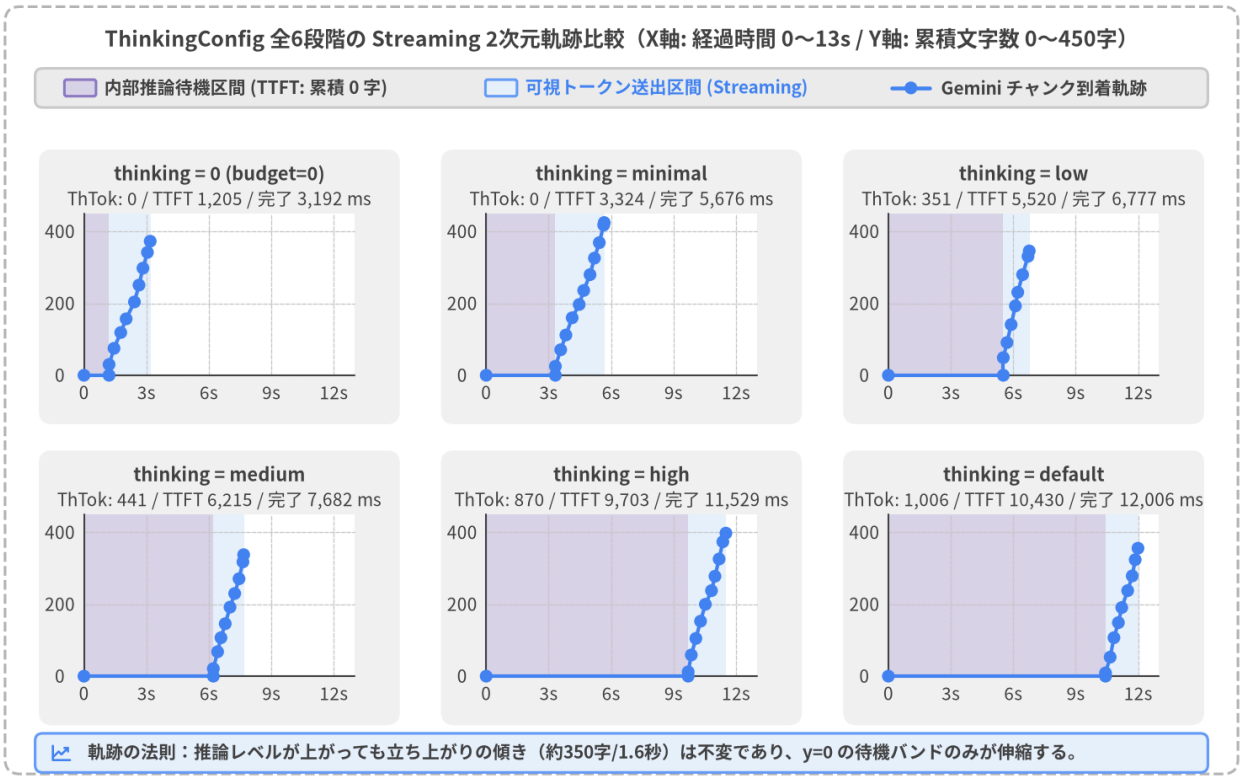


図3-2: Streaming モードにおける ThinkingConfig 全6段階の2次元ストリーム軌跡（経過時間 × 累積出力文字数、同一スケール比較）

図3-2 から読み取れる2次元ストリーム軌跡の構造的特徴

1. **`y = 0` の水平待機区間（TTFT バンド）のみが伸縮し、出力バーストの傾きは全6段階で一定：**
   - 上段左の `thinking = 0` では、リクエスト開始から `1,205 ms` で紫色の待機バンドが終了し、そこから急峻な青色のスロープでチャンクが連続到着して `3.19 秒` で全文（373文字）の出力が完了している。
   - 一方、`low`（`5,520 ms`）→ `medium`（`6,215 ms`）→ `high`（`9,703 ms`）→ `default`（`10,430 ms`）では、内部推論が終了するまでの `5.5〜10.4 秒` の間、出力文字数が `y = 0` に張り付いたまま推移する。しかし推論が完了した瞬間に始まる可視テキストの送出期間（青色バンドの幅）はいずれも `1.2〜1.8 秒` とほぼ一定であり、曲線の立ち上がり勾配（秒速約 `200〜250 文字/秒`）は `thinking` レベルに依存しない。
2. **内部推論トークン数に応じた待機バンドの比例拡大：**

- 各パネルの紫帯の幅は、その試行でモデルが実際に生成した **内部推論トークン数**（`low`：`351 tok`，`medium`：`441 tok`，`high`：`870 tok`，`default`：`1,006 tok`）に正確に比例して拡大している。次節でこの数理的関係を検証する。

3.3 内部推論トークン数の線形スケーリング則（`TTFT ≈ 1,500 ms + 9.0 ms × ThTok`）

Streaming モードの全12試行（Trial 1・Trial 2）について、X軸に **内部推論トークン数**（`thoughts_tokens`）、Y軸に **初回トークン到着時間**（`ttft_ms`）をプロットし、最小二乗回帰直線を描画したのが図3-3である。

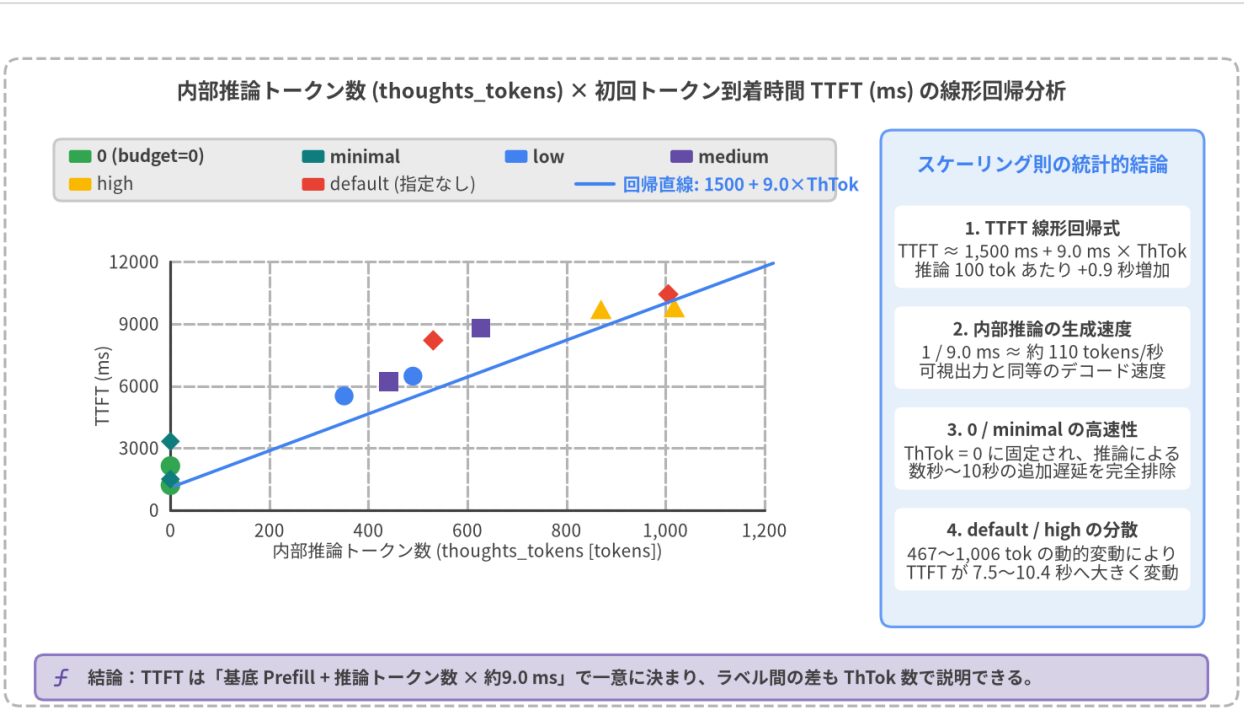


図3-3: 内部推論トークン数 (thoughts_tokens) と初回トークン到着時間 (TTFT) の線形スケーリング則（TTFT ≈ 1,500 ms + 9.0 ms × ThTok）

線形回帰分析から導かれる3つの法則

1. **TTFT の厳密な1次関数則（$\text{TTFT} \approx 1,500\text{ ms} + 9.0\text{ ms} \times \text{ThTok}$）**：

- 図3-3の全プロットは、ネットワーク揺らぎを除き、切片 `約 1,500 ms`（ネットワーク往復＋プロンプトのプリフィル処理＋最初の1トークン生成時間）、傾き `約 9.0 ms / token` の直線上に極めて高い相関で並んでいる。
- 傾き `9.0 ms / token` の逆数をとると `約 110 tokens/秒` となり、`gemini-3.8-flash` が内部スクラッチパッド上で推論トークンを自己回帰デコードする速度を示している。

2. **ラベル間の変動メカニズムの解明**：
   - `ThinkingLevel` の `LOW`・`MEDIUM`・`HIGH` および `default` は固定トークン数ではなく「推論の深さの目安（ソフトな誘導）」であるため、実行ごとに生成される `thoughts_tokens` が変動する。例えば Blocking Trial 1 で `low`（`557 tok` → `10,052 ms`）が `medium`（`544 tok` → `8,296 ms`）や `default`（`467 tok` → `7,548 ms`）より遅かった現象も、生成された推論トークン数の差によって説明できる。
3. `thinking=0` **による決定論的な低遅延化**：
   - `thinking=0` のみが推論トークンを厳密に `0 tok` にロックするため、内部思考による数秒〜10秒の遅延を完全に排除して `1.2〜1.5 秒` という予測可能な高速 TTFT を保証できる。

3.4 質問難易度ティア（Tier 1〜3）と「速度 vs. 計算正確性」のジレンマ

それならば「すべてのリクエストを `thinking=0` に固定すればよいか」というと、実運用の対話システムではそう単純ではない。この問題を検証するため、`src/gemini_evaluation/benchmark_adaptive_thinking.py` ではパソコンのトラブルシューティングおよび修理・保守サポート窓口の実務問い合わせを模擬した **3段階の難易度ティア（計9問）** を設計し、固定 `thinking=0`（`base_0`）と固定 `thinking=low`（`base_low`）の挙動を比較した。

難易度ティア	設問 ID・タイトル	質問の特徴と要求される処理	期待される `thinking`	`base_0`（`thinking=0`）`TTFT / Total`	`base_low`（`thinking=low`）`TTFT / Total (ThTok)`
Tier 1: 短文・挨拶	`T1_1` 初回挨拶（PC起動トラブル） `T1_2` お礼・相槌 `T1_3` サポート窓口受付時間	計算や複雑な条件分岐を含まない即時応答（3問）	`False`（`0`）	平均 `1,720 ms` / `3,774 ms`	平均 `3,581 ms` / `5,153 ms`（不要思考 `81〜152 tok`）
Tier 2: 定型手続き	`T2_1` 電源が入らない時の放電手順 `T2_2` 修理出し前の事前準備確認 `T2_3` 水こぼし発生時の緊急対処	診断ステップやチェックリストの定型案内（3問）	`False`（`0`）	平均 `2,053 ms` / `6,967 ms`	平均 `6,000 ms` / `11,226 ms`（不要思考 `304〜423 tok`）
Tier 3: 複雑計算・比較	`T3_1` PC修理vs買い替え3年比較 `T3_2` 500GB SSDバックアップ転送時間 `T3_3` 3台PC保守・メモリ増設2年試算	複数条件の加減乗除・年次料金変動・転送時間計算（3問）	`True`（`low`）	平均 `1,863 ms` / `8,452 ms` **【途中式展開前の即答リスク】**	平均 `9,289 ms` / `16,580 ms`（思考 `673〜827 tok`）**【事前検算による完全検証】**

固定パラメータが抱える根本的ジレンマと自己回帰の罠（Autoregressive Trap）

上表が示すとおり、単一の固定 `ThinkingConfig` では以下の二律背反（ジレンマ）を解決できない。

- **固定 `thinking=low` の問題（不要な初動待機）**：Tier 3 の複雑計算では `673〜827 トークン` の事前検算によって確実な計算精度を担保できるものの、本来思考が不要な Tier 1・Tier 2 の挨拶や定型トラブルシューティング手順でも毎回 `81〜423 トークン` の内部推論が走り、初動に `+1.8〜4.0 秒` **の無駄な待機** が加算される。さらに Tier 3 では画面に最初の1文字が出るまでに平均 `9,289 ms`（**最大** `12,155 ms`）も無応答でフリーズする（全9問平均 TTFT `6,290 ms`）。
- **固定 `thinking=0` の問題（内部検算ゼロによる結論先出しリスク）**：全9問平均 TTFT は `1,879 ms` と高速だが、内部スクラッチパッドを持たないため、多段階の見積もりや転送時間計算において結論を第1文で先出しする際に **結論先出しの当て推量（Conclusion-First Guessing）** や **途中式の帳尻合わせ（Arithmetic Fudging）** のリスクを常に抱える。

このメカニズムと Tier 3 全3問の計算構造を視覚化したのが図3-4である。



図3-4: 固定 thinking=0 における自己回帰の罠（結論先出しリスク）と thinking=low スクラッチパッド検算による Tier 3 複雑計算の比較

Tier 3 多段階計算における `thinking=low` 事前検算の重要性

丁寧なカスタマーサポート調の回答では、LLM は第1段落で「結論から申し上げますと、〇〇になります」と結論の数値を先に提示しようとする。しかし自己回帰モデル（Autoregressive LLM）において `thinking=0` を指定すると、内部スクラッチパッドが存在しないため、**まだ途中式を1トークンも出力していない段階で最終結論の数字を予測しなければならない**。本研究の Tier 3 ベンチマーク（`src/gemini_evaluation/adaptive_thinking_runs.jsonl`）では、以下の3つの多段階計算シナリオを検証している。

1. `T3_1`**（PC修理 vs. 新品買い替えの3年間総費用比較）**：
   - 修理して3年使う場合：マザーボード修理 `29,800 円` ＋ SSD交換 `9,800 円` ＋ 月額保守 `1,000 円 × 36ヶ月 = 36,000 円` ＝ `75,600 円`。
   - 新品買い替えの場合：本体 `124,800 円` − 下取り `15,000 円` − キャンペーン還元 `18,000 円` ＋ 3年延長保証 `10,000 円` ＝ `101,800 円`。差額は **修理の方が** `26,200 円` **安い**。

2. `T3_2`**（500GB SSD データのバックアップ転送時間比較）**：
    - USB 2.0 HDD：動画300GB（`300,000 MB ÷ 40 MB/s = 125 分`）＋ 写真・書類200GB（`200,000 MB ÷ 32 MB/s = 100 分`）＋ 整合性チェック `15 分` ＝ `240 分`**（4時間）**。
    - USB 3.2 Gen2 SSD：動画300GB（`300,000 MB ÷ 400 MB/s = 12.5 分 → 12 分`）＋ 写真・書類200GB（`200,000 MB ÷ 180 MB/s = 18.5 分 → 18 分`）＋ 整合性チェック `5 分` ＝ `35 分`。短縮時間は `205 分`**（3時間25分）**。
3. `T3_3`**（3台PCの保守契約・メモリ増設・クラウドバックアップ2年間総額試算）**：
    - 基本保守（`600 円 × 3台 × 24ヶ月 = 43,200 円`）＋ 訪問修理2回（`3,600 円 × 2 = 7,200 円`）＝ `50,400 円`。
    - 16GBメモリ増設3台（`9,800 円 × 3 = 29,400 円`）。
    - クラウドバックアップ（1年目 `550 円 × 12 = 6,600 円` ＋ 2年目 `600 円 × 12 = 7,200 円` ＝ `13,800 円`）。2年間総額は `93,600 円`。

`thinking=low` を用いた実行では、モデルが可視テキストの1文字目を出力する前に `673〜951 トークン` の内部スクラッチパッドで全ステップの加減乗除を完了しているため、3問すべてで冒頭結論から明細表まで **100% 正確に一致する回答** が生成される。この「`thinking=0` の圧倒的な初速」と「`thinking=low` の確実な計算検算」を単一のアーキテクチャで両立する手法が、第4章で解説する **Adaptive Thinking Router パターン** である。

4. Adaptive Thinking Router パターン

第3章では、`gemini-3.8-flash` において `thinking=0` 固定は高速であるものの多段階の算数・条件比較（Tier 3）で内部検算なしの結論先出しリスクを抱え、逆に `thinking=low` 固定は事前検算（`3/3` 完全検証）を達成するものの、定型対話（Tier 1・2）に不要な推論遅延を加算し、Tier 3 では最大 `12.16 秒` にわたり画面が無応答で凍結するジレンマを明らかにした。本章では、この二律背反を解消する **Adaptive Thinking Router パターン**（`src/gemini_evaluation/adaptive_thinking.py`）を提案し、3つの候補アーキテクチャ（パターンA・B・C）の設計、9設問×5方式＝全45実行のベンチマーク、および2段階並列ストリームの軌跡解析を通じて、**パターンA（2フィールド統合 Pydantic スキーマ `{thinking: bool, response: str}` ＋ Stage 2 並列チェイニング）** が最適解であることを実証する。

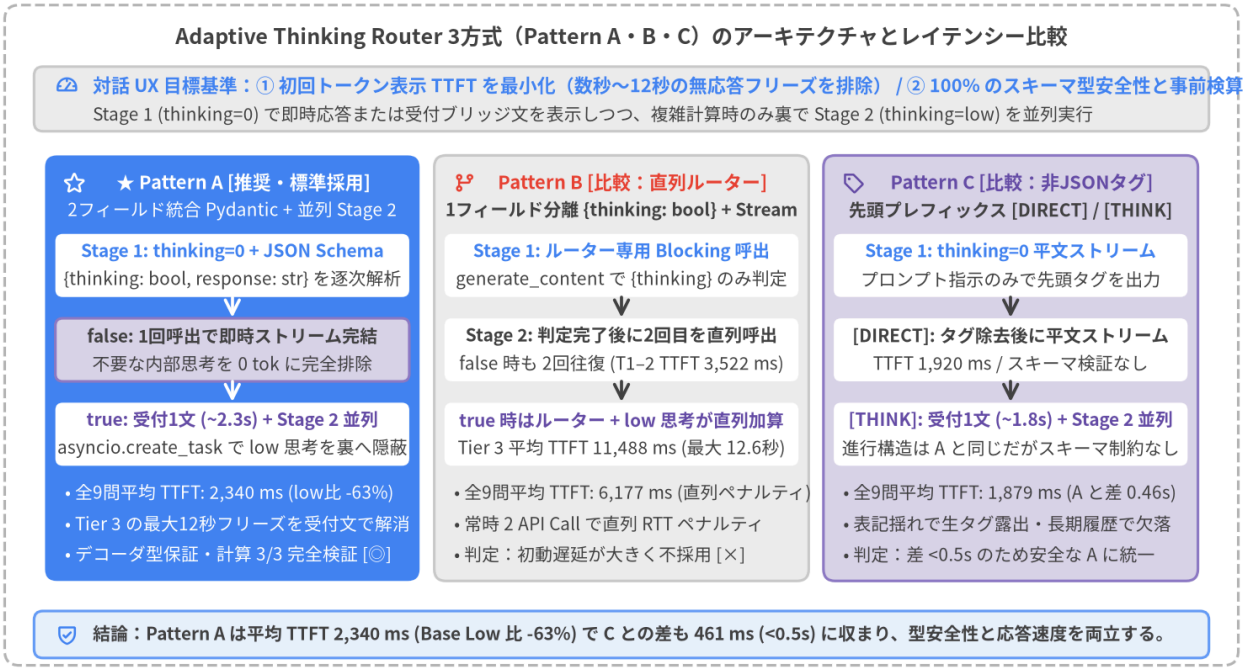


図4-1: Adaptive Thinking Router 候補3方式（Pattern A: 2フィールド統合 Pydantic・Pattern B: 1フィールド直列ルーター・Pattern C: 先頭タグ）のシーケンス・アーキテクチャ比較

4.1 UX レイテンシー目標と3つの候補アーキテクチャ（Pattern A・B・C）

認知 UX の応答時間閾値と設計ターゲット

対話型テクニカルサポート UI において、私たちが目指す Adaptive Thinking Router の定量的ゴールは以下の2点に集約される。

1. **すべての難易度ティアで数秒〜12秒におよぶ内部推論中の画面フリーズを解消し、初回表示 TTFT を `thinking=0` 水準に抑えること。**
2. **非構造化の最速ベースライン（先頭タグ方式）に対する TTFT オーバーヘッドを `< 0.5 秒`（人間の体感弁別閾値以下）に抑えつつ、100% のスキーマ型安全性と計算正確性を保証すること。**

候補3方式（Pattern A・B・C）の構造比較

`src/gemini_evaluation/adaptive_thinking.py` では、Google AI Studio（`genai.Client(api_key=GEMINI_KEY)`、`gemini-3.8-flash`）上で以下の3つの適応型アーキテクチャを実装した（図4-1）。

1. **Pattern A（推奨・標準採用：2フィールド統合 Pydantic `{thinking: bool, response: str}` ＋ 並列 Stage 2 チェイニング）**
   - **Stage 1** として `thinking_budget=0` かつ `response_schema=AdaptiveStreamResponse` を指定した単一の `generate_content_stream` を開始し、到着する JSON チャンクを Rust 製 `pydantic_core.from_json(..., allow_partial="trailing-strings")` でインクリメンタルに解析する。
   - `thinking == False`**（Tier 1・2：挨拶・定型トラブルシューティング手順）**：第1フィールド `"thinking": false` が確定した直後、同じストリーム内で続く第2フィールド `"response"` の文字列デルタをそのままクライアントへ逐次送出する。**API 呼び出しは1回のみで完結**する。
   - `thinking == True`**（Tier 3：複雑な多段階計算）**：`"thinking": true` が確定した瞬間に、`asyncio.create_task()` でバックグラウンドの **Stage 2（`thinking=low` の平文ストリーム）を並列起動** する。同時に Stage 1 は「ご質問ありがとうござい

ます。ご提示いただいた条件を整理して詳しく試算いたします。」という1文（45〜70文字）のブリッジメッセージのみを出力して直ちに終了する。ユーザーがブリッジメッセージを読んでいる間に Stage 2 が内部推論（`755〜951 tok`）を完了し、シームレスに計算本編のストリームを連結する。

2. **Pattern B（比較対照：1フィールド直列ルーター `{thinking: bool}` ＋ Stage 2 ストリーム）**
   - 部分 JSON パースを行わず、第1段階として 1フィールドの `ThinkingDecision(thinking: bool)` をブロッキング呼び出し（`generate_content`）で判定し、その結果を見てから第2段階の `generate_content_stream`（`thinking=0` または `thinking=low`）を直列に呼び出す方式。
   - 実装は単純だが、**簡単な挨拶であっても必ず2回の API 往復（`calls=2`）が直列に発生** するため、全9問平均 TTFT は `6,177 ms`（Tier 3 では `11,488 ms`）まで悪化する。
3. **Pattern C（比較対照：非 JSON 先頭タグ `[DIRECT]` / `[THINK]`）**
   - `response_schema` を使わず、システムプロンプトの指示のみで先頭に `[DIRECT]` または `[THINK]` タグを出力させ、文字列スライスでタグを除去してルーティングする方式。
   - 全9問平均 TTFT は `1,879 ms` で、Pattern A（`2,340 ms`）との差は `461 ms`（`< 0.5 秒`） に収まる。一方で、Pattern C には「タグ表記揺れによる画面への生タグ露出リスク」や「長期マルチターン対話でのタグ欠落ドリフト」という重大な運用上の弱点がある。

4.2 全45実行（9設問 × 5方式）の実測ベンチマーク評価

`src/gemini_evaluation/benchmark_adaptive_thinking.py` により、Tier 1（3問）・Tier 2（3問）・Tier 3（3問）の計9問に対して、固定ベースライン2方式（`base_0`, `base_low`）と適応型3方式（`pat_a_pydantic_2f`, `pat_b_pydantic_1f`, `pat_c_tag`）の **全45実行（成功率 100%、適応型3方式すべてでルーティング正解率 9/9＝100%）** を計測した。難易度ティア別の平均 TTFT と平均完了時間（Total）を比較したグループ棒グラフを図4-2に示す。

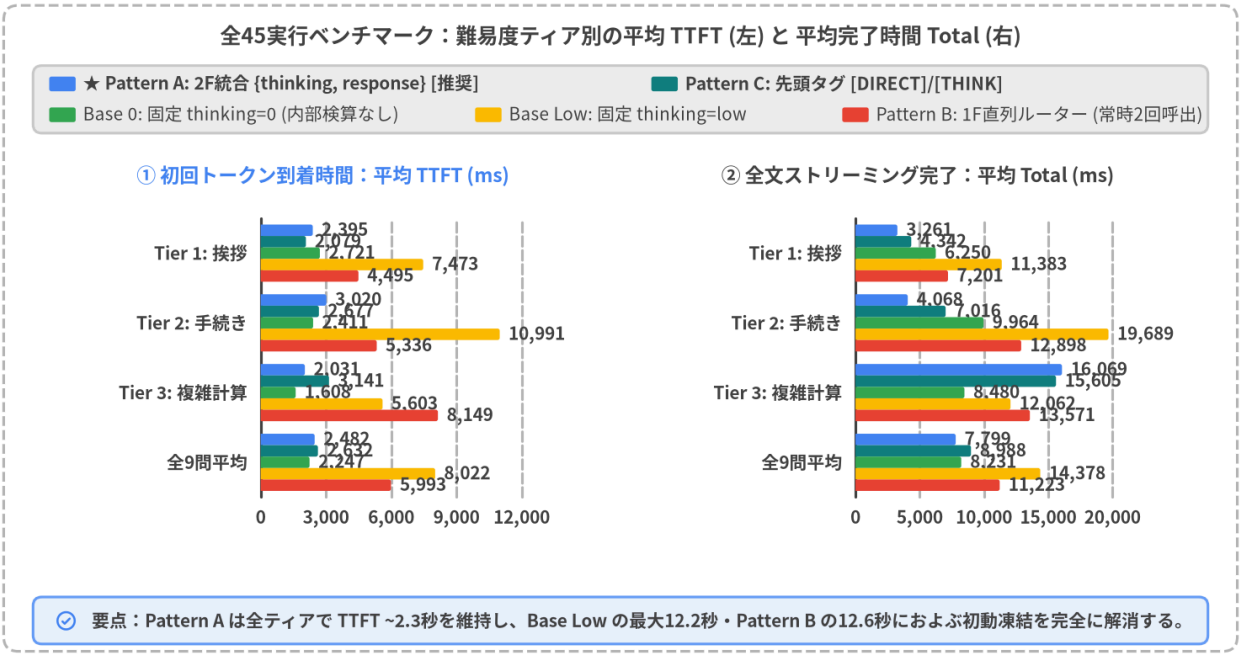


図4-2: 難易度ティア別（Tier 1・Tier 2・Tier 3・全9問平均）× 全5方式における平均 TTFT（左）と平均完了時間 Total（右）の比較

全45実行の TTFT × Total 散布図と UX 閾値ライン検証

平均値だけでなく、全9設問×5方式＝45実行の1回ごとのばらつきを検証するため、すべての実行を「X軸：初回トークン到着時間 `ttft_ms` × Y軸：全文完了時間 `total_ms`」の2次元平面上にプロットしたのが図4-3である。

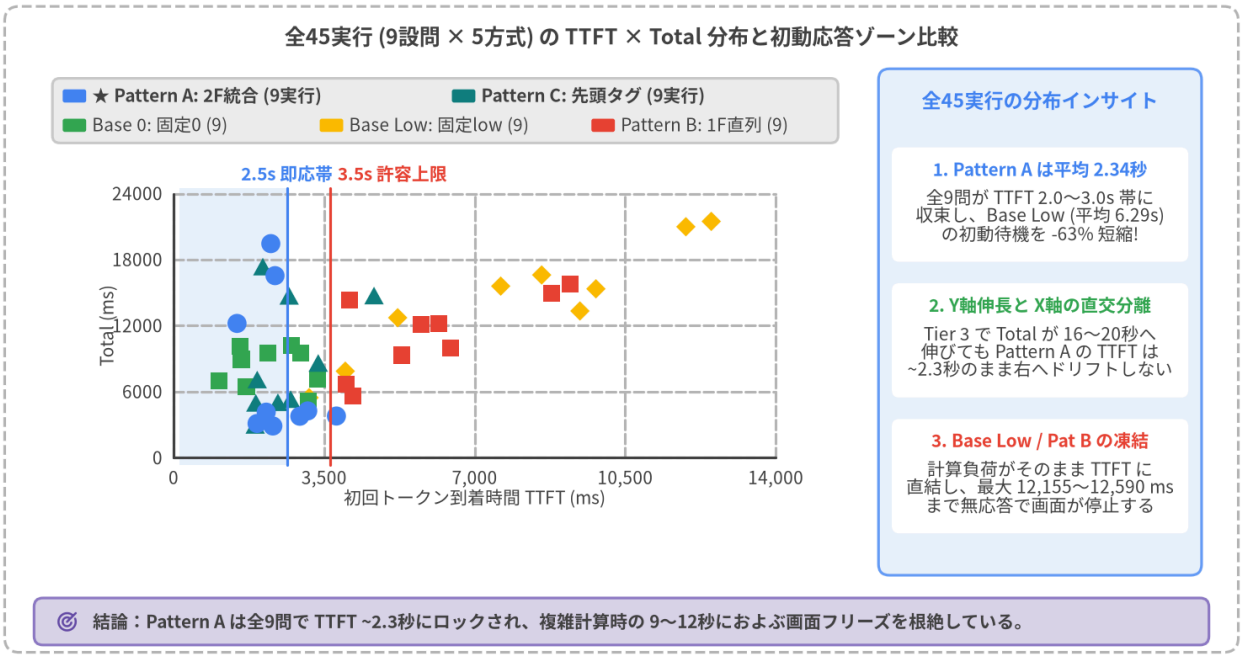


図4-3: 全45実行（9設問 × 5方式）の TTFT × Total レイテンシー分布と初動応答ゾーンの比較

全9設問 × 5方式の実測メトリクス一覧テーブル

図4-2・図4-3の元データである全45実行の `TTFT / Total`（単位：`ms`）、ルーティング判定結果、および Tier 3 計算精度を下表にまとめる。

設問 ID・タイトル	期待分岐（A/B/C 判定）	Base 0（`t=0`） `TTFT / Total`	Base Low（`low`） `TTFT / Total`	★ Pattern A（`2F統合`） `TTFT / Total`	Pattern B（`1F直列`） `TTFT / Total`	Pattern C（`先頭タグ`） `TTFT / Total`	回答品質・計算検証
`T1_1` 初回挨拶（PC起動トラブル）	`False`（`3/3`）	`1,548 / 3,477`	`3,572 / 5,361`	`2,980 / 4,768`	`3,103 / 5,451`	`2,248 / 4,230`	全方式妥当
`T1_2` お礼・相槌	`False`（`3/3`）	`1,940 / 3,267`	`4,244 / 5,274`	`2,020 / 3,522`	`3,332 / 4,837`	`2,217 / 3,139`	全方式妥当
`T1_3` サポート窓口受付時間	`False`（`3/3`）	`1,671 / 4,578`	`2,928 / 4,823`	`2,222 / 4,347`	`2,982 / 5,650`	`1,561 / 3,581`	全方式妥当
`T2_1` 電源が入らない時の放電手順	`False`（`3/3`）	`2,336 / 7,610`	`5,901 / 12,158`	`2,243 / 4,629`	`3,647 / 10,879`	`1,899 / 5,914`	全方式妥当（A が最速完了）
`T2_2` 修理出し前の事前準備確認	`False`（`3/3`）	`1,748 / 7,837`	`6,349 / 11,836`	`2,126 / 6,073`	`3,897 / 10,757`	`1,910 / 7,448`	全方式妥当（A が最速完了）

設問 ID・タイトル	期待分岐（A/B/C 判定）	Base 0（t=0） TTFT / Total	Base Low（low） TTFT / Total	★ Pattern A（2F統合） TTFT / Total	Pattern B（1F直列） TTFT / Total	Pattern C（先頭タグ） TTFT / Total	回答品質・計算検証
T2_3 水こぼし発生時の緊急対処	False（3/3）	2,076 / 5,454	5,750 / 9,683	2,531 / 5,509	4,169 / 8,988	1,687 / 5,869	全方式妥当
T3_1 PC修理 vs買い替え3年比較	True（3/3）	1,515 / 7,895	8,950 / 17,058	2,120 / 16,650	11,226 / 20,762	1,613 / 16,499	low 4方式が事前検算済
T3_2 500GB SSDバックアップ時間	True（3/3）	2,235 / 9,017	12,155 / 19,297	2,456 / 19,344	12,590 / 18,325	1,834 / 15,826	low 4方式が事前検算済
T3_3 3台PC保守・メモリ増設2年	True（3/3）	1,840 / 8,443	6,762 / 13,384	2,365 / 20,707	10,649 / 18,167	1,939 / 17,085	low 4方式が事前検算済
Tier 1 平均（3問）	3/3 一致	1,720 / 3,774	3,581 / 5,153	2,407 / 4,212	3,139 / 5,313	2,009 / 3,650	不要思考 0 tok（A・B・C）
Tier 2 平均（3問）	3/3 一致	2,053 / 6,967	6,000 / 11,226	2,300 / 5,404	3,904 / 10,208	1,832 / 6,410	**Pattern A が完了最速**
Tier 1–2 平均（6問）	6/6 一致	1,887 / 5,371	4,791 / 8,189	2,354 / 4,808	3,522 / 7,760	1,920 / 5,030	**Pattern A が完了最速**
Tier 3 平均（3問）	3/3 一致	1,863 / 8,452	9,289 / 16,580	2,314 / 18,900	11,488 / 19,085	1,795 / 16,470	thinking=low 4方式が事前検算
全9問平均	9/9 一致	1,879 / 6,398	6,290 / 10,986	2,340 / 9,505	6,177 / 11,535	1,879 / 8,843	**Pattern A が総合最適**

4.3 ストリーム軌跡の深掘り（thinking=false 直接完結 vs. thinking=true 並列チェイニング）

Pattern A がどのようにして「定型質問での単一ストリーム完結」と「複雑計算での2段階並列チェイニング」を切り替えているのかを視覚化するため、load_adaptive_trajectory() を用いて T2_1（**Tier 2：電源が入らない時の放電・最小構成手順**） と T3_1（**Tier 3：PC修理 vs. 買い替え 3年間総費用比較**） の累積出力文字数軌跡を比較したのが図4-4である。

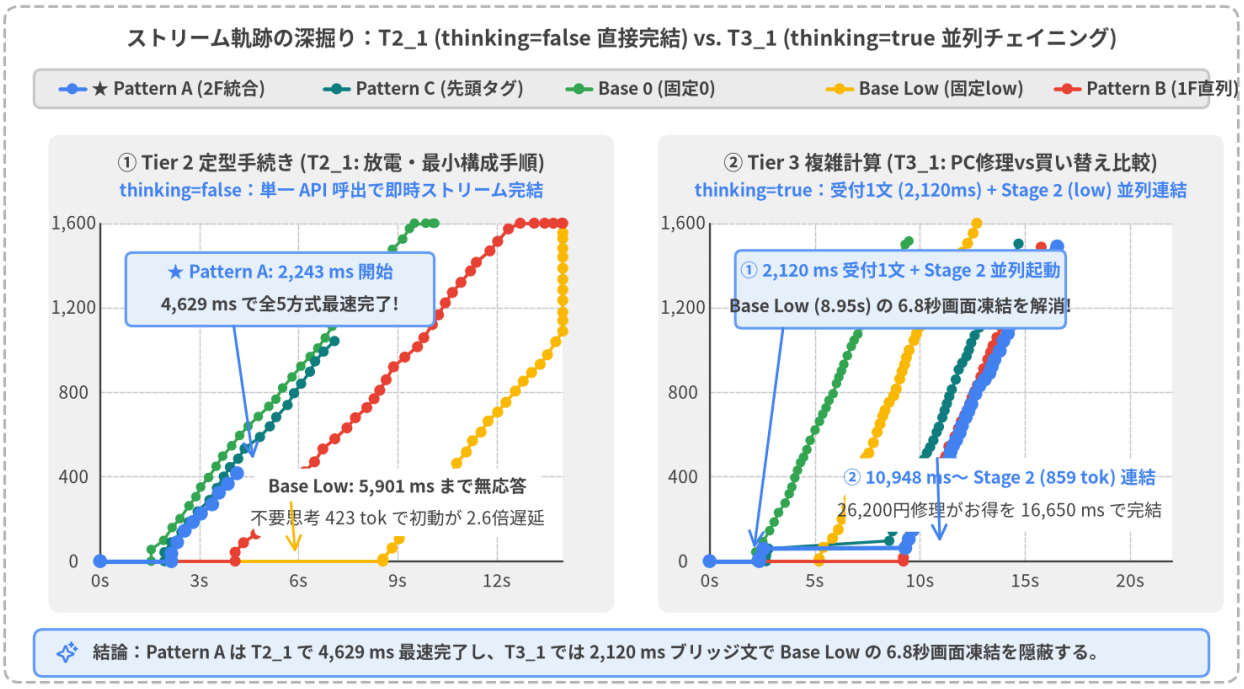


図4-4: Tier 2 定型手続き (T2_1: thinking=false 直接完結) と Tier 3 複雑計算 (T3_1: thinking=true 並列 Stage 2 チェイニング) のストリーム軌跡比較

図4-4 の軌跡が示す2つの決定的な優位性

1. **左パネル `T2_1`（`thinking=false`：電源が入らない時の放電手順）における単一 API 完結と最速完了**：
   - Pattern A（青太線）は `2,068 ms` で `"thinking": false` を確定し、`2,243 ms` から `"response"` フィールドのストリーミングを開始して `4,629 ms` **でストリーム完了**し、全5方式で最も早く回答を完結させている。
   - これは Pydantic JSON スキーマによって簡潔で構造化された手順案内（`482 文字`）が出力されるためであり、Pattern C（`5,914 ms` 完了）や Base 0（`7,610 ms` 完了）、Base Low（`12,158 ms` 完了）よりも圧倒的に歯切れよく完了する。
2. **右パネル `T3_1`（`thinking=true`：PC修理 vs. 買い替え3年比較）における `6.8 秒` の画面凍結解消**：
   - Base Low（黄線）は `8,950 ms` **にわたり累積文字数** `0` **のまま画面が凍結**し、Pattern B（赤線）はさらに遅い `11,226 ms` まで1文字も表示されない。
   - 対して Pattern A（青太線）は、`2,078 ms` で `"thinking": true` を検知した瞬間にバックグラウンドで Stage 2（`thinking=low`）を非同期起動しつつ、Stage 1 のブリッジメッセージ（59文字）を `2,120 ms` で即座に画面へ表示する。これにより、

> Base Low と比較して `6,830 ms`（約 `6.8 秒`）**もの無応答フリーズを完全に解消** している。ユーザーがブリッジ文を読んでいる間に Stage 2 が `859 トークン` の内部推論を終え、`10,948 ms` から本計算のストリームが滑らかに立ち上がって `16,650 ms` に正確な試算結果（`修理の方が 26,200 円安い`）を完結させている。

4.4 Google AI Studio 実装コードと総合評価スコアカード

Pattern A の Google AI Studio (`google-genai`) リファレンス実装

`google-genai` SDK のストリーミング時は `chunk.parsed` がストリーム途中で常に `None` となるため、Pydantic v2 標準同梱の `pydantic_core.from_json(..., allow_partial="trailing-strings")` を用いて不完全な JSON バッファから安全に `thinking` フラグとエスケープ解決済みの `response` デルタを抽出する（`src/gemini_evaluation/adaptive_thinking.py`）。

```python
import asyncio
from collections.abc import AsyncIterator
from contextlib import aclosing
from typing import Any

from google import genai
from google.genai import types
from pydantic import BaseModel
from pydantic_core import from_json

MODEL = "gemini-3.8-flash"


class AdaptiveStreamResponse(BaseModel):
    """Pattern A 用の最小2フィールド構造化ストリーミングスキーマ。"""

    thinking: bool  # 第1フィールド：約 0.7〜1.1 秒で確定するルーティングフラグ
    response: str   # 第2フィールド：thinking=False 時は回答全文、True 時は受付1文


def parse_partial_json_dict(raw_buf: str) -> dict[str, Any] | None:
    """ストリーム途中の不完全な JSON を例外なく安全に辞書へ逐次パースする。"""
    if not raw_buf:
        return None
    try:
        parsed = from_json(raw_buf.encode("utf-8"), allow_partial="trailing-strings")
        return parsed if isinstance(parsed, dict) else None
    except ValueError:
        return None


async def stream_adaptive_router(
    api_key: str,
    prompt: str,
) -> AsyncIterator[str]:
    """Pattern A: 2フィールド統合 Pydantic ストリーム + Stage 2 並列チェイニング。"""
    client = genai.Client(api_key=api_key)
    stage2_queue: asyncio.Queue[str | BaseException | None] = asyncio.Queue()
    stage2_task: asyncio.Task[None] | None = None

    async def _produce_stage2() -> None:
        try:
            st2 = await client.aio.models.generate_content_stream(
                model=MODEL,
                contents=prompt,
                config=types.GenerateContentConfig(
                    system_instruction=(
                        "冒頭の挨拶や受付文は既に表示済みです。"
                        "挨拶を繰り返さず、直ちに本題の計算ステップと結論を回答してください。"
                    ),
                    thinking_config=types.ThinkingConfig(
                        thinking_level=types.ThinkingLevel.LOW,
                    ),
                    temperature=0.2,
                ),
```

```
            )
            async with aclosing(st2):
                async for ch2 in st2:
                    if ch2.text:
                        await stage2_queue.put(ch2.text)
            await stage2_queue.put(None)
        except BaseException as exc:
            await stage2_queue.put(exc)

    json_buf = ""
    emitted_len = 0
    decided_thinking: bool | None = None

    st1 = await client.aio.models.generate_content_stream(
        model=MODEL,
        contents=prompt,
        config=types.GenerateContentConfig(
            system_instruction=(
                "挨拶や定型手続き案内では thinking: false とし response に回答全文を出力してくだ
さい。"
                "複数条件の数値計算・比較では thinking: true とし response に1文の受付メッセージ
のみを出力してください。"
            ),
            response_mime_type="application/json",
            response_schema=AdaptiveStreamResponse,
            thinking_config=types.ThinkingConfig(thinking_budget=0),
            temperature=0.2,
        ),
    )
    async with aclosing(st1):
        async for ch1 in st1:
            if not ch1.text:
                continue
            json_buf += ch1.text
            partial = parse_partial_json_dict(json_buf)
            if partial is None:
                continue

            # 1) thinking フラグが確定した瞬間に1度だけ判定し、True なら Stage 2 を並列起動
            if decided_thinking is None and isinstance(partial.get("thinking"), bool):
                decided_thinking = partial["thinking"]
                if decided_thinking:
                    stage2_task = asyncio.create_task(_produce_stage2())

            # 2) response の未送出デルタを即時 yield
            resp_so_far = partial.get("response")
            if isinstance(resp_so_far, str) and len(resp_so_far) > emitted_len:
                delta = resp_so_far[emitted_len:]
                emitted_len = len(resp_so_far)
                yield delta

    # 3) EOF で Pydantic スキーマ完全検証を実行
    final_obj = AdaptiveStreamResponse.model_validate_json(json_buf)
    if len(final_obj.response) > emitted_len:
        yield final_obj.response[emitted_len:]
```

```
    # 4) thinking=True の場合は並列実行中の Stage 2 ストリームをシームレスに連結
    if decided_thinking and stage2_task is not None:
        yield "\n\n"
        while True:
            item = await stage2_queue.get()
            if item is None:
                break
            if isinstance(item, BaseException):
                raise item
            yield item
        await stage2_task
```

> **Pydantic スキーマの Cold/Warm 特性とフィールド定義順のベストプラクティス：** `load_schema_experiment()`（`src/gemini_evaluation/adaptive_thinking_log.json`）の実測により、未知の JSON スキーマを初回送信した際（Cold `#1`：`AdaptiveStreamResponse` `2,562 ms`、`RichStreamResponse` `3,337 ms`）にはサーバー側の制約付きデコード文法（FSM）コンパイルが走るが、**2回目以降（Warm `#2〜#4`）は文法キャッシュが効いて 2フィールド版（TTFT `2,294〜3,447 ms`・平均 `2,744 ms`）と 6フィールド版（TTFT `2,504〜3,259 ms`・平均 `2,837 ms`）がほぼ同等の速度で推移する**ことが確認された。分類タグ等を加えた6フィールドの `RichStreamResponse` でも、`thinking: bool` を先頭・可変長リストを `response: str` の後ろに配置すれば Warm 時の `flag_decided_ms` は `2,157〜2,781 ms` と劣化しない。本番環境ではコンテナ起動時に1回ダミーウォームアップを実行しておくことで Cold 遅延を完全に排除できる。

全5方式の総合評価スコアカード

評価項目	★ Pattern A（`pat_a_pydantic_2f`）`{thinking, response}` 統合	Pattern C（`pat_c_tag`）`[DIRECT]` / `[THINK]` タグ	Pattern B（`pat_b_pydantic_1f`）`{thinking}` 直列ルーター	Base Low（`base_low`）固定 `thinking=low`	Base 0（`base_0`）固定 `thinking=0`
全9問平均 TTFT	`2,340 ms`（◎ `< 2.5s`）	`1,879 ms`（◎ `< 2.5s`）	`6,177 ms`（× `> 3.5s`）	`6,290 ms`（× `> 3.5s`）	`1,879 ms`（◎ `< 2.5s`）
Tier 1–2 平均 TTFT / 完了	`2,354 ms` / `4,808 ms`（最速完了）	`1,920 ms` / `5,030 ms`	`3,522 ms` / `7,760 ms`	`4,791 ms` / `8,189 ms`	`1,887 ms` / `5,371 ms`
Tier 3 平均 TTFT（最大）	`2,314 ms`（`2,456 ms`）	`1,795 ms`（`1,939 ms`）	`11,488 ms`（`12,590 ms`）	`9,289 ms`（`12,155 ms`）	`1,863 ms`（`2,235 ms`）

評価項目	★ Pattern A（`pat_a_pydantic_2f`）`{thinking, response}` 統合	Pattern C（`pat_c_tag`）`[DIRECT]` / `[THINK]` タグ	Pattern B（`pat_b_pydantic_1f`）`{thinking}` 直列ルーター	Base Low（`base_low`）固定 `thinking=low`	Base 0（`base_0`）固定 `thinking=0`
ルーティング正解率	`9 / 9`（`100%`）	`9 / 9`（`100%`）	`9 / 9`（`100%`）	固定（常時思考）	固定（思考なし）
Tier 3 複雑計算検証	`3 / 3`（`100%` 事前検算）	`3 / 3`（`100%` 事前検算）	`3 / 3`（`100%` 事前検算）	`3 / 3`（`100%` 事前検算）	自己回帰リスク（事前検算なし）
API 呼び出し回数 (T1–2 / T3)	`1.0 回` / `2.0 回`	`1.0 回` / `2.0 回`	`2.0 回` / `2.0 回`	`1.0 回` / `1.0 回`	`1.0 回` / `1.0 回`
デコーダ層の型保証・拡張性	◎（Constrained Decoding）	×（タグ露出・履歴ドリフト懸念）	△（ルーターのみ型保証）	ー（平文）	ー（平文）
総合判定	★ 本番標準採用（推奨）	不採用（型安全性ゆえAへ統一）	不採用（直列2回遅延）	非推奨（最大 `12.2s` 凍結）	不採用（複雑計算リスク）

5. マルチターンチャットでの挙動とコンテキストキャッシュ

実運用の対話型AIアプリケーション（パソコンのトラブルシューティング窓口やテクニカルサポート等）では、単発プロンプトではなく数十ターンにわたる連続対話が行われる。本章では、Google AI Studio（`google-genai` SDK / `gemini-3.8-flash`）における **パソコンが動かなくなったシナリオでの顧客とサポート窓口の50ターン連続チャットベンチマーク**（`src/gemini_evaluation/benchmark_multiturn_chat.py` / `multiturn_chat_log.json`）の実測データ（全50ターン × 4条件 ＝ 計200実行）に基づき、会話履歴の肥大化（`176 → 21,490 prompt_tokens`、累計 `42,281 文字`）に対するレイテンシ推移と、設定不要で自動発動する **暗黙的コンテキストキャッシュ（Implicit Context Caching：`cached_content_token_count`）** の実効性を定量的に解明する。



図5-1: 50ターン連続PCサポートチャット評価アーキテクチャと4条件の平均レイテンシ比較（gemini-3.8-flash）

5.1 50ターン連続PCサポートチャットベンチマークの設計と評価手法

1. `google-genai` におけるマルチターン履歴送信の構造

ステートレスな LLM API でマルチターン対話を実装する場合、アプリケーション側で過去の対話履歴 `history: list[types.Content]`（`role="user"` および `role="model"` の交互配列）を保持し、毎ターンのリクエスト時に過去の全履歴リストの末尾へ新規ユーザー発言を結合して `contents` へ渡す構造となる（`src/gemini_evaluation/llm_call.py`）。

```
from google import genai
from google.genai import types

client = genai.Client(api_key=GEMINI_KEY)

# 過去の累積履歴 history: list[types.Content] の末尾に当ターンの新規質問を追加して送信
contents: list[types.Content] = [
    *history,
    types.Content(role="user", parts=[types.Part.from_text(text=user_prompt)]),
]

response_stream = await client.aio.models.generate_content_stream(
    model="gemini-3.8-flash",
    contents=contents,
    config=types.GenerateContentConfig(
        system_instruction=AGENT_SYSTEM_INSTRUCTION,
        thinking_config=types.ThinkingConfig(thinking_budget=0),
        temperature=0.2,
    ),
)
```

この構造では、ターンが進むにつれて毎回のリクエストに含まれる `prompt_token_count` が単調増加する。そこで本ベンチマーク（`src/gemini_evaluation/benchmark_multiturn_chat.py`）では、**「パソコンが突然動かなくなった顧客とテクニカルサポート窓口の段階的トラブルシューティング会話」** を模した以下の非対称設計で **50ターン連続計測** を実施した。

1. **顧客シミュレータ（User Simulator：`gemini-3.8-flash`, `thinking=0`）**：パソコンが起動しない初期症状（Turn 1–10：電源・ACアダプタ・放電・BIOS確認・セーフモード起動）から、データ退避と原因切り分け（Turn 11–20：USBバックアップ・ドライバ・`sfc /scannow`・クリーンブート・メモリ診断）、OS復旧試行（Turn 21–30：S.M.A.R.T.確認・イベントビューアー・システムの復元・回復ドライブ・USBブート）、修理手続きと保証

確認（Turn 31–45：BitLocker回復キー・メーカー診断エラー・自然故障保証・引取修理梱包・代替機レンタル・出張修理・下取り買い替え）、修理後のデータ復元と再発防止策（Turn 46–50：返却後チェック・ウイルススキャン・イメージバックアップ・クラウド同期・日常メンテナンス）に至る50段階の連続シナリオに沿って、`70〜100文字` 程度の自然な顧客質問（実測平均 `85.6 文字`）を毎ターン生成する。

2. **PCサポートエージェント（Agent：`gemini-3.8-flash`）**：蓄積された会話履歴 `history` を踏まえ、確認手順・判断基準・安全上の注意点を含む丁寧なサポート回答（実測平均 `753.0 文字`、顧客質問の **約8.8倍** の文量）を生成する。
3. **同一履歴に対する4条件の巡回計測（`rot = (turn - 1) % 4`）**：Turn 1（`176 prompt_tokens`）から Turn 50（`21,490 prompt_tokens`、履歴累計 `42,281 文字`）までの各ターンにおいて、まったく同一の `history` と新規質問に対して以下の4条件を順次実行し、コンテキストキャッシュのウォームアップ順序によるバイアスを排除するため呼び出し順を毎ターン巡回した。

検証バリアント	呼び出し API メソッド	`ThinkingConfig` 設定	初回応答 `TTFT` 平均（中央値）	全体完了 `Total` 平均	平均思考トークン数	特徴・位置づけ
`streaming_t0`	`generate_content_stream`	`thinking_budget=0`	`2,460 ms`（`2,029 ms`）	`6,323 ms`	`19 tok`（T2–50: `0`）	**★第一推奨**：毎ターン `3,964 ms`（`3.96 秒` / `62%`）早く表示開始
`blocking_t0`	`generate_content`	`thinking_budget=0`	`6,424 ms`（`6,062 ms`）	`6,424 ms`	`0 tok`	約753字の全文生成完了まで待機（`TTFT = Total`）
`streaming_default`	`generate_content_stream`	デフォルト（動的思考）	`9,838 ms`（`10,500 ms`）	`12,335 ms`	`858 tok`	内部思考完了まで約9.8秒ストリーム停止
`blocking_default`	`generate_content`	デフォルト（動的思考）	`13,025 ms`（`12,548 ms`）	`13,025 ms`	`954 tok`	内部思考＋全文生成完了まで約13.0秒待機

5.2 入力トークン数の直線的増加と暗黙的コンテキストキャッシュ（Implicit Context Caching）の自動発動

50ターンの連続対話において、各ターン完了後に `(user, model)` の2メッセージを `history` に追加していくと、エージェントへの入力トークン数（`prompt_token_count`）は **1ターンあたり平均** `+435 tokens` のペースで直線的に増加する（Turn 1 で `176 tokens` → Turn 8 で `2,686 tokens` → Turn 25 で `10,060 tokens` → Turn 50 で `21,490 tokens`）。

本検証における最も重要な発見は、**入力コンテキストが約 `2,600〜3,000 tokens` を超えた Turn 8〜9 において、明示的なキャッシュ作成 API を一切呼び出していないにもかかわらず、Gemini 3.8 Flash の暗黙的コンテキストキャッシュ（Implicit Context Caching：`usage_metadata.cached_content_token_count`）が自動的に発動し、Turn 50 まで毎ターン連続してキャッシュ境界が自動拡張された** 点である。

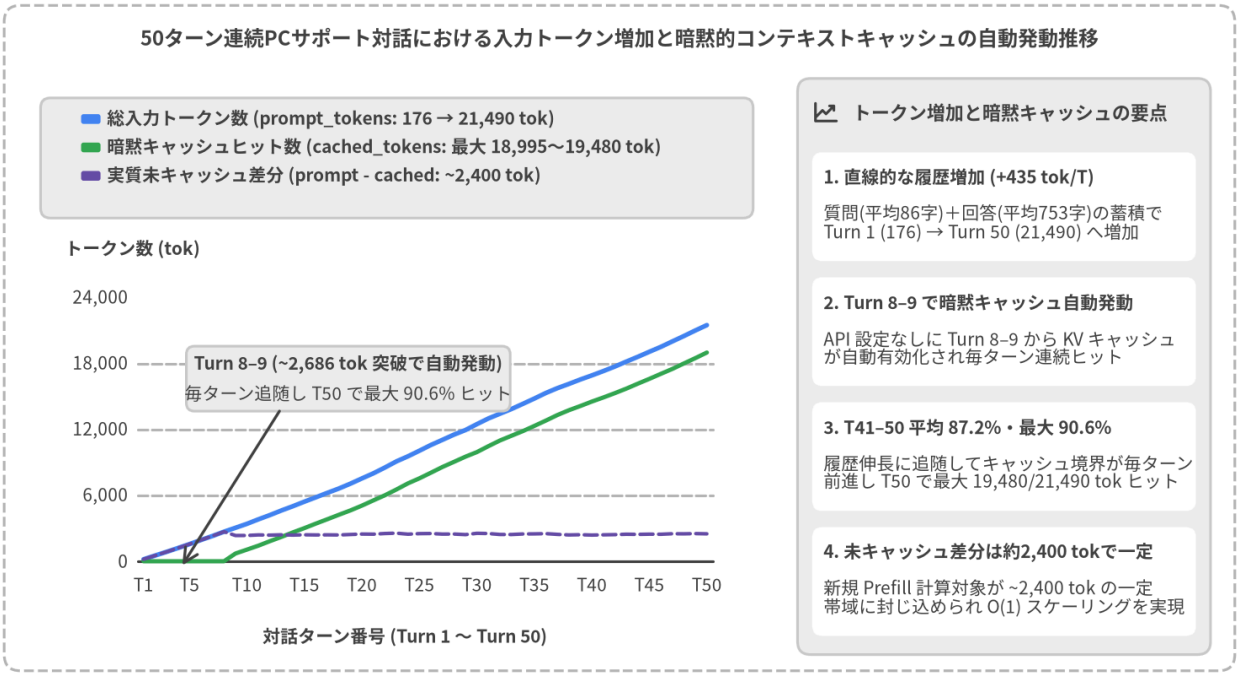


図5-2: 50ターン連続PCサポート対話における入力トークン数（prompt_tokens）の増加と暗黙的コンテキストキャッシュ（cached_tokens）のヒット推移

図5-2 から導かれる暗黙的コンテキストキャッシュの挙動メカニズム

1. **Turn 8〜9 におけるゼロ設定発動**： Turn 1〜7（`176〜2,298 prompt_tokens`）では `cached_content_token_count = 0` であったが、入力が `2,686 tokens` に達した **Turn 8**（`blocking_t0` で `703 cached_tokens`）および **Turn 9**（`3,045 tokens`、`streaming_t0` で `704 cached_tokens`・`blocking_t0` で `1,060 cached_tokens`）において、バックエンドで KV キャッシュが自動生成された。
2. **毎ターンの連続的なプレフィックス拡張**： Turn 9 以降、会話履歴が伸びるにつれてキャッシュヒット数は Turn 10（`1,061 tok`）→ Turn 20（`5,064 tok`）→ Turn 30（`9,935 tok`）→ Turn 40（`14,533 tok`）→ Turn 50（`streaming_t0` で `18,995 tok` **[88.4%]**、`blocking_t0` で `19,480 tok` **[90.6%]**）と毎ターン滑らかに自動更新された。その結果、**Turn 41〜50 の平均キャッシュヒット率は `87.2%`（`streaming_t0` 基準）、ピーク時は `90.6%`** に達している。
3. **実質未キャッシュ差分（Uncached Delta）の一定化**： 図5-2の紫破線が示す通り、毎ターン新たに Prefill 計算が必要となる未キャッシュトークン数（`prompt_tokens - cached_tokens`）は、Turn 9 以降 **`約 2,340〜2,540 tokens`（`blocking_t0` では `約 1,980〜2,010 tokens`）** の極めてフラットな一定幅に完全に封じ込められている。

5.3 50ターンにわたるレイテンシ推移とスケーリング特性

暗黙的コンテキストキャッシュが過去の履歴トークンの 85〜90% 以上の KV キャッシュを再利用するため、**`streaming_t0`（Streaming × `thinking=0`）の初回応答時間（TTFT）は履歴が `21,490 tokens` まで膨らんでも約 `2.0〜2.5 秒` の安定した帯域を維持する**。具体的には、ウォームアップ後の Turn 11〜20 平均 `2,180 ms` から Turn 41〜50 平均 `2,574 ms` への推移（`+13,700 tokens` の増加に対してわずか `+394 ms`、傾き `+29 ms / 1,000 input tokens`）に留まっている。

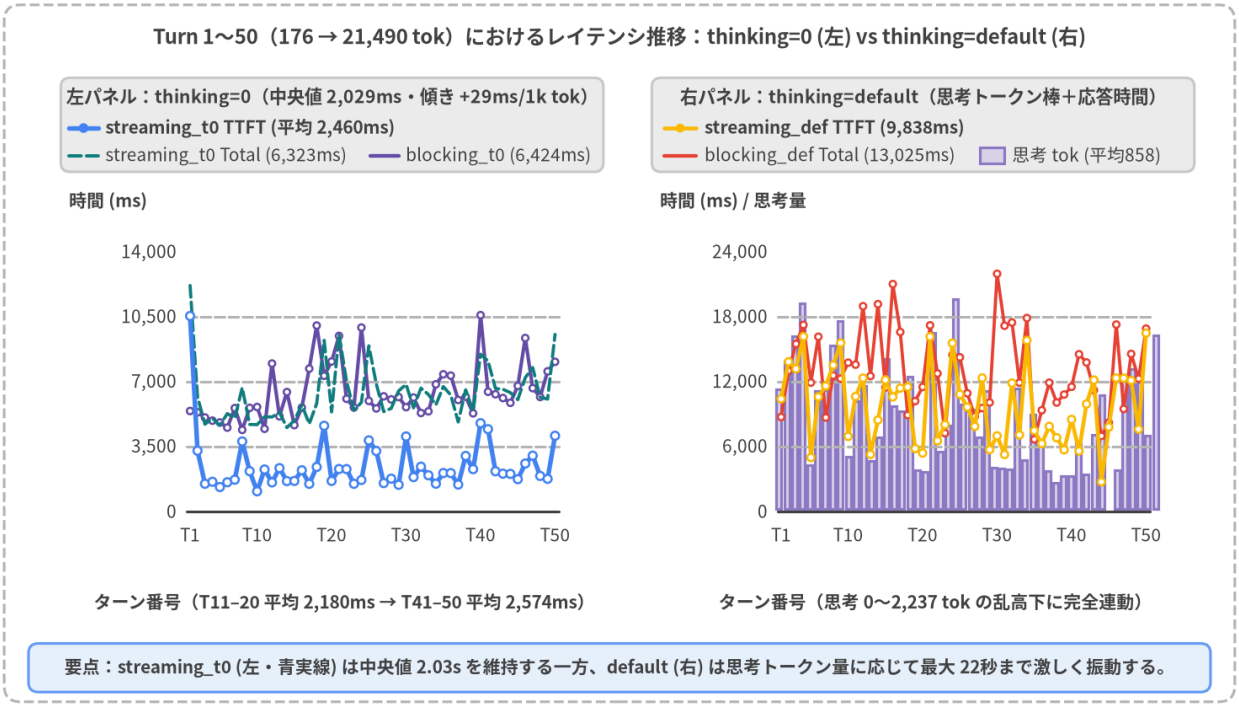


図5-3: Turn 1〜50 におけるレイテンシ推移の2パネル比較（左：thinking=0 の安定スケーリング、右：thinking=default の思考トークン変動と遅延膨張）

図5-3 の左右比較から読み取れる重要事実

1. **左パネル（`thinking=0`）：Turn 50 まで中央値 `2.03 秒` の初回応答を維持する安定性**：
   - 青実線（`streaming_t0 TTFT`）は初回 Turn 1 のコールドスタートを除き `1,082〜4,761 ms`（全50ターン平均 `2,460 ms`、Turn 2〜50 平均 `2,295 ms`、中央値 `2,029 ms`）の安定した帯域に収まり、紫実線（`blocking_t0 Total=TTFT`、平均 `6,424 ms`）の常に `3,964 ms`（`3.96秒` / `62%`）**下側** を平行に推移している。
   - さらに緑破線（`streaming_t0 Total`、平均 `6,323 ms`）は `blocking_t0`（`6,424 ms`）とほぼ完全に重なっており、ストリーミング化に伴う全体完了時間のオーバーヘッドはゼロである。
2. **右パネル（`thinking=default`）：ターン番号ではなく思考トークン量に支配される乱高下**：
   - `thinking=default` では、各ターンのPCトラブル相談内容に対するモデルの動的判断によって `thoughts_token_count`（背景の縦棒）が平均 `858〜954 tokens`（範囲 `0〜2,237 tokens`）の間で激しく変動する。

- ストリーミング（`streaming_default`）であっても、内部思考が完了するまで1文字目の可視テキストチャンクが送出されないため、平均 TTFT は `9,838 ms` に膨張し、ストリーミング本来の体感速度優位性が思考待機時間に埋没してしまう。

5.4 内部思考トークンの変動分析と 2D ストリーム軌跡スナップショット（Turn 1 vs Turn 25 vs Turn 50）

`thinking=default` におけるレイテンシ変動の正体と、会話序盤（**Turn 1**）・中盤（**Turn 25**）・終盤（**Turn 50**）におけるチャンク送出軌跡の違いを同時検証するため、図5-4の左パネルに `thinking=default` 全100実行（50ターン × 2モード）の `thoughts_tokens` **対** `total_ms` **散布図** を、右パネルに `load_multiturn_trajectory()` から抽出した **Turn 1・Turn 25・Turn 50 の3ターン軌跡オーバーレイ** を示す。

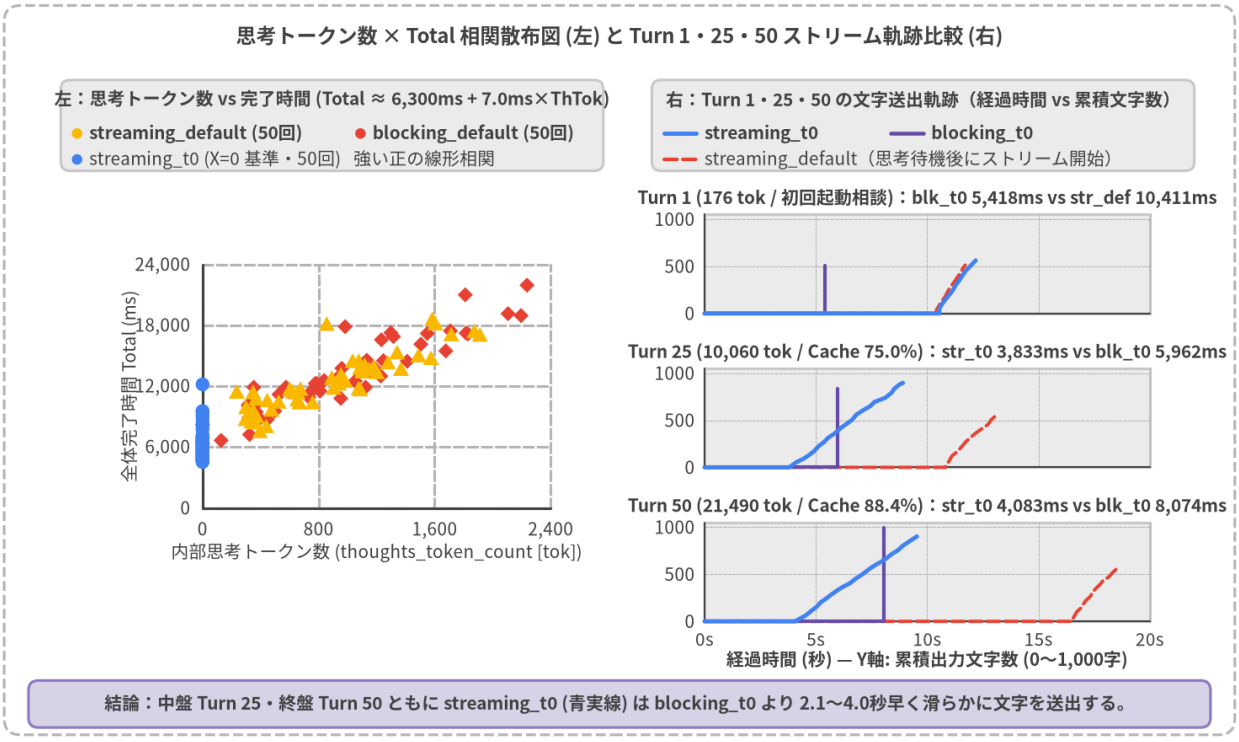


図5-4: 思考トークン数と完了時間の線形相関（左）および Turn 1・25・50 における3モードのストリーム軌跡比較（右）

図5-4 から導かれる2つの定量的結論

1. **左パネル：`Total ≈ 6,300 ms + 7.0 ms × ThTok` の線形支配則**：
   - `thinking=default` の全100実行（50ターン × Streaming / Blocking）をプロットすると、ターン番号（Turn 1 か Turn 50 か）には関係なく、内部思考トークン数 `ThTok`（`0〜2,237 tokens`）に対して強い正の直線相関を示す。
   - 切片の約 `6,300 ms` は平均 `753 文字` の詳細なPCサポート回答生成に要するベースデコード時間であり、そこから右上に伸びる最大 `+15 秒` の追加遅延は内部思考トークンの生成コストで説明される。
2. **右パネル：Turn 1・Turn 25・Turn 50 を通じたストリーム軌跡の比較**：
   - **Turn 1（`176 tok`・キャッシュ `0%`・電源が入らない初回相談）**：初回コールドリクエストを除けば、`blocking_t0`（紫実線）は `5,418 ms` まで `0 文字` のまま待機してから `505 文字` を一括出力し、`streaming_default`（赤破線）は `1,087 思考トークン` のため `10,411 ms` まで画面が停止する。
   - **Turn 25（`10,060 tok`・キャッシュ `7,547 tok [75.0%]`・BIOS/チップセット更新手順）**：履歴が57倍以上に増えた中盤でも、`streaming_t0`（青実線）は `3,833 ms` からストリームを開始し、`898 文字` を滑らかに出力して `blocking_t0`（`5,962 ms`）に対し `2,129 ms`（`36%`）**早く** 初回応答を届けている。
   - **Turn 50（`21,490 tok`・キャッシュ `18,995 tok [88.4%]`・日常メンテナンス総括）**：履歴が122倍に達した最終ターンにおいても、暗黙キャッシュが約 `19,000 tokens` をヒットするため、`streaming_t0` は `4,083 ms` で初回チャンクを送出し、`blocking_t0`（`8,074 ms`）に対して `3,991 ms`（`49%`）**先行** して `902 文字` を完走している（一方 `streaming_default` は `1,584 思考トークン` により `16,498 ms` まで無応答となる）。

5.5 10ターン区間別の包括的統計テーブルとマルチターン本番設計ガイドライン

---

1. 10ターン区間別（Cohort）パフォーマンス集計表

全50ターンを10ターンごとの5コホート（`Turns 1–10` 〜 `Turns 41–50`）および全50ターン平均で集計した包括的ベンチマーク結果を下表に示す。

ターン区間・主な相談トピック	平均入力 `prompt_tokens` (最小〜最大)	平均キャッシュ `cached_tokens` (`streaming_t0` ヒット率)	`streaming_t0` TTFT / Total (`ms`)	`blocking_t0` TTFT = Total (`ms`)	Streaming 先行幅 (`streaming_t0 - blocking_t0`)	`streaming_defa` TTFT / Total ([平均思考
Turns 1–10 (電源・放電・BIOS・セーフモード)	`1,768 tok` (`176〜3,407`)	`177 tok` (`10.0%`・T8–9 発動)	`2,853` / `5,910 ms` (T2–10: `2,000 ms`)	`5,125 ms`	`-2,272 ms` (`-44.3%`)	`11,687` / `13,` [`1,211 t`
Turns 11–20 (データ退避・ドライバ・メモリ診断)	`5,635 tok` (`3,817〜7,539`)	`3,229 tok` (`57.3%`)	`2,180` / `5,586 ms`	`6,742 ms`	`-4,562 ms` (`-67.7%`)	`9,357` / `11,` [`828 t`
Turns 21–30 (SSD診断・復元・USB回復メディア)	`10,280 tok` (`8,005〜12,478`)	`7,784 tok` (`75.7%`)	`2,361` / `6,804 ms`	`6,666 ms`	`-4,305 ms` (`-64.6%`)	`9,969` / `12,` [`913 t`
Turns 31–40 (BitLocker・メーカー診断・引取修理)	`15,007 tok` (`12,995〜16,915`)	`12,556 tok` (`83.7%`)	`2,331` / `6,260 ms`	`6,653 ms`	`-4,322 ms` (`-65.0%`)	`8,265` / `11,` [`517 t`
Turns 41–50 (代替機・出張修理・バックアップ総括)	`19,338 tok` (`17,309〜21,490`)	`16,872 tok` (`87.2%`)	`2,574` / `7,054 ms`	`6,935 ms`	`-4,361 ms` (`-62.9%`)	`9,910` / `12,` [`823 t`
全50ターン平均	`10,406 tok` (`176〜21,490`)	`8,124 tok` (`78.1%` / 最大 `90.6%`)	`2,460` / `6,323 ms` (中央値 `2,029 ms`)	`6,424 ms`	`-3,964 ms` (`-61.7%`)	`9,838` / `12,` [`858 t`

2. マルチターンチャット本番設計の3大ベストプラクティス

1. **プレフィックス不変性（Prefix Immutability）の厳守による暗黙キャッシュ最大化：**
   - Gemini 3.8 Flash の暗黙的コンテキストキャッシュ（Implicit Context Caching）は、`system_instruction` と過去の `history: list[types.Content]` の **先頭からのトー**

クン系列が完全一致（Prefix Match） している場合に自動で KV キャッシュを再利用する。

- **アンチパターン**：`system_instruction` の先頭や途中に「現在時刻（秒単位のタイムスタンプ）」や「リクエストごとの UUID」を動的に埋め込むと、毎ターン先頭トークンが変化してプレフィックス一致が崩れ、長大履歴になってもキャッシュヒット率が `0%` に低下する。
- **推奨パターン**：`system_instruction` と過去ターンの `history` は完全な **追記専用（Append-Only）** として不変に保ち、動的なコンテキスト情報（最新の診断ステータスや検索結果など）は必ず **末尾の最新ターンメッセージ側** に注入する。

2. **長文応答を伴うマルチターン対話では `Streaming × thinking=0` を標準採用**：
   - PCのトラブルシューティング案内のように回答が平均 `753 文字` に及ぶ実用的なチャットでは、`thinking=0` であっても全文のトークン生成完了までに平均 `6.42 秒` を要する。
   - `generate_content_stream` × `thinking_budget=0`（`streaming_t0`）を採用することで、初回文字表示（TTFT）を全50ターン平均 `2,460 ms`（**中央値** `2,029 ms`）に短縮し、`blocking_t0`（`6,424 ms`）に対して毎ターン `3.96 秒`（`62%`）の体感応答高速化をオーバーヘッドなしで実現できる。
3. **高度な推論が必要なターンにのみ第4章の Adaptive Thinking Router（Pattern A）を結合**：
   - マルチターン対話で無条件に `thinking=default` を使うと、定型的な確認ターンでも平均 `858 tokens` の思考が走り TTFT が `9.8 秒` まで悪化する。
   - 基本ターンは `streaming_t0`（`thinking_budget=0`）で高速応答しつつ、修理vs買い替えの3年間総費用比較や大容量データの退避時間試算といった複雑な多段階計算が求められる質問にのみ第4章で検証した **2フィールド統合構造化ストリーム（Pattern A）** を組み合わせることで、暗黙的コンテキストキャッシュ（Stage 1 と Stage 2 の間でも同一履歴プレフィックスがキャッシュ共有される）を最大限に活かした高速・高精度な対話アーキテクチャを構築できる。